

Pós-Graduação em Ciência de Dados e Analytics

Sprint de Engenharia de Dados



Cálculo e estudo do Índice de Desenvolvimento Social para o município de Belo Horizonte, capital de Minas Gerais, com base nos dados do Censo Demográfico de 2022.

Daniel de Luna Marques

Matrícula 4052026000228

Rio de Janeiro, 2026.

Índice de Figuras

Figura 1 - Screenshot da página de Downloads dos resultados do Universo agregados por setores censitários do Censo Demográfico de 2022	11
Figura 2 – Overview do Catalog Explorer: IDS_MVP e os schemas criados para as camadas Bronze, Silver e Gold.....	12
Figura 3 - Overview do Volumes na camada bronze, com os dados ingeridos em sua forma bruta.	13
Figura 4- Parte 1 do código de ingestão de dados a partir da camada Bronze e a criação das tabelas deltas nessa mesma camada.	14
Figura 5 - Parte 2 do código de ingestão de dados a partir da camada Bronze e a criação das tabelas delta nessa mesma camada.....	14
Figura 6 - Linhas dos arquivos do Censo Demográfico de 2022 que foram ingeridos. .	15
Figura 7 - Código de renomeação de variáveis das tabelas alfabetizacao e dom2, oriundas da camada Bronze.....	16
Figura 8 - Código de criação da tabela setor-bairro-favela, a partir filtragem da variável NM_MUN para o município de Belo Horizonte.	17
Figura 9- Filtrando os setores censitários as tabelas temáticas a partir da relação setor-bairro do município de Belo Horizonte.	18
Figura 10 - Seleção de variáveis nas tabelas	20
Figura 11 - Verificação da tipagem dos dados	21
Figura 12 - Parte 1 Script de Identificação de Valores Não Numéricos	22
Figura 13 - Parte 2 Script de Identificação de Valores Não Numéricos	22
Figura 14 - Parte 3 Script de Identificação de Valores Não Numéricos	23
Figura 15 - Parte 4 Script de Identificação de Valores Não Numéricos	23
Figura 16 - Parte 5 Script de Identificação de Valores Não Numéricos	24
Figura 17- Parte 6 Script de Identificação de Valores Não Numéricos	24
Figura 18 - Resultados da Identificação de valores Não Numéricos.....	25
Figura 19 - Parte 1 do código feito para remover os valores "X" das variáveis e passá-las para int ou float.....	26
Figura 20 - Parte 2 do código feito para remover os valores "X" das variáveis e passá-las para int ou float.....	27

Figura 21- Código de gravação das tabelas na camada Silver	27
Figura 22 - Parte 1 do Código de Preenchimento de Metadados das Tabelas na Camada Silver.....	28
Figura 23 - Parte 2 do Código de Preenchimento de Metadados das Tabelas na Camada Silver.....	28
Figura 24 - Parte 1 do Código de substituição de valores nulos por 0.	29
Figura 25 - Parte 2 do Código de substituição de valores nulos por 0.	30
Figura 26 - Parte 3 do Código de substituição de valores nulos por 0.	30
Figura 27 - Camada Bronze, com as tabelas delta.....	32
Figura 28 - Overview do schema Silver e suas tabelas, com os metadados preenchidos na forma de comentários	33
Figura 29- Overview da tabela de Alfabetização na camada Silver e seus metadados.	34
Figura 30 - Overview da tabela Domicílios 1 na camada Silver e seus metadados.	34
Figura 31 - Overview da tabela Domicílios 2 na camada Silver e seus metadados.	35
Figura 32 - Overview da tabela de Renda na camada Silver e seus metadados.	35
Figura 33 - Overview da tabela de Relação Setor-Bairro na camada Silver e seus metadados.	36
Figura 34 - Overview do schema Gold e suas tabelas, com os metadados preenchidos na forma de comentários.	37
Figura 35 - Parte 1 do Overview da tabela de Cestas de Variáveis agregados por setores censitários, na camada Gold.	38
Figura 36 – Parte 2 do Overview da tabela de Cestas de Variáveis agregados por setores censitários, na camada Gold.	38
Figura 37 - Parte 3 do Overview da tabela de Cestas de Variáveis agregados por setores censitários, na camada Gold.	39
Figura 38 - Overview da tabela de Cestas de Variáveis agregados por setores censitários, na camada Gold.	39
Figura 39 - Parte 1 código da junção das tabelas da camada Silver.	41
Figura 40 - Parte 2 código da junção das tabelas da camada Silver.	41
Figura 41 - Parte 1 Código de construção de Cesta de Variáveis do IDS.....	44
Figura 42 - Parte 2 Código de construção de Cesta de Variáveis do IDS.....	45

Figura 43- Parte 3 Código de construção de Cesta de Variáveis do IDS.....	45
Figura 44 - Parte 1 do Código de construção da tabela de valores máximos e mínimos para a normalização.	46
Figura 45 - Parte 2 do Código de construção da tabela de valores máximos e mínimos para a normalização.	47
Figura 46 – Parte 1 do código de normalização e verificação de valores globais de máximos e mínimos.....	48
Figura 47 – Parte 2 do código de normalização e verificação de valores globais de máximos e mínimos.....	49
Figura 48 - Código de gravação das tabelas na camada Gold.	50
Figura 49 - Indicadores normalizados e IDS por setores censitários	50
Figura 50 - Indicadores normalizados e IDS por bairros.....	51
Figura 51 - Indicadores normalizados e IDS por favelas ou comunidades urbanas.....	51
Figura 52 - Indicadores normalizados e IDS Geral para o município de Belo Horizonte	52
Figura 53 - Código de consulta em SQL para responder a primeira pergunta.	54
Figura 54 - Código de consulta em SQL para responder a segunda pergunta.....	56
Figura 55- Código de consulta em SQL para responder a terceira pergunta.	58
Figura 56- Código de consulta em SQL para responder a quarta pergunta.	59
Figura 57 – Parte 1 do código de consulta em SQL para responder a quinta pergunta..	60
Figura 58 - Parte 2 do código de consulta em SQL para responder a quinta pergunta...	60
Figura 59 - Código de consulta em SQL para responder a sexta pergunta.....	62
Figura 60 - Código de consulta em SQL para responder a sétima pergunta.	63

Sumário

1.	Contexto de Negócios e Perguntas	6
1.1.	Contexto de Negócios.....	6
1.2.	Contexto dos dados brutos.....	6
1.2.1.	Fonte dos Dados	6
1.2.2.	Licença de Uso	8
1.3.	Perguntas de Negócio	9
2.	Carga dos Dados	11
3.	Qualidade de Dados.....	16
4.	Modelagem e Catálogo de Dados.....	32
4.1.	Catálogo de Dados.....	32
4.2.	Modelagem	40
7.	Pipeline de Dados	53
8.	Análise de Dados.....	54
9.	Autoavaliação	64

1. Contexto de Negócios e Perguntas

1.1. Contexto de Negócios

O Índice de Desenvolvimento Social é calculado pelo Instituto Pereira Passos -- instituto de pesquisa da prefeitura do Rio de Janeiro -- a fim de medir o grau de desenvolvimento social de uma determinada área geográfica em comparação com outras, as quais são, além do próprio município, recortes ou subdivisões intramunicipais, como setores censitários, bairros, favelas.

Oficialmente, o IPP só calcula o IDS para o município do Rio de Janeiro. Contudo, como o Índice baseia-se inteiramente em variáveis dos Censo Demográfico, é perfeitamente possível aplicar a sua metodologia para analisar o cenário de desenvolvimento social em outros municípios. A sua metodologia pode ser avaliada nesta nota técnica, publicada no DATA.RIO, o portal de dados abertos da prefeitura do município do Rio de Janeiro: [Nota Técnica 44 – Adaptação Metodológica do IDS](#).

Este pipeline toma como cenário hipotético a necessidade da administração municipal de Belo Horizonte, capital do estado de Minas Gerais, de realizar uma análise do desenvolvimento social da população com base na metodologia do IDS.

Logo, **o objetivo geral deste projeto é calcular o IDS para o município de Belo Horizonte (MG) e para seus diferentes recortes geográficos intramunicipais, incluindo setores censitários, bairros e favelas, além do próprio município, possibilitando a análise do desenvolvimento social em diferentes níveis territoriais.**

1.2. Contexto dos dados brutos

1.2.1. Fonte dos Dados

Para viabilizar o cálculo do Índice de Desenvolvimento Social (IDS) no município de Belo Horizonte, utilizam-se os microdados e agregados oficiais do **Censo Demográfico 2022**, produzidos e disponibilizados pelo **Instituto Brasileiro de Geografia e Estatística (IBGE)**.

Os dados utilizados encontram-se agregados no menor nível territorial de divulgação pública do Censo: o **setor censitário**. Isso permite a análise tanto na escala de setor quanto a agregação hierárquica para bairros, favelas/comunidades urbanas e o município como um todo.

Todos os arquivos foram extraídos do repositório público de estatísticas do IBGE, na seção dedicada aos [Agregados por Setor Censitário em formato CSV](#).

Cabe ressaltar que os arquivos de dados agregados por setores censitários contêm um volume expressivo de variáveis. A listagem exaustiva de todas elas neste documento extrapolaria o escopo do trabalho e incluiria dados não utilizados neste MVP. Para eventuais consultas sobre o conjunto completo de variáveis, [disponibiliza-se o link para o dicionário de dados do Censo Demográfico 2022](#).

Os arquivos selecionados para o projeto contemplam as seguintes bases temáticas e metadados:

1. basico.csv (Agregados por Setores Censitários - Básico):

- Fornece a malha cadastral e geográfica de referência, contendo a identificação do setor censitário (CD_SETOR), Unidade da Federação (NM_UF), Município (NM_MUN), Bairro (NM_BAIRRO) e classificação do setor em Favelas e Comunidades Urbanas (NM_FCU).

2. alfabetizacao.csv (Agregados por Setores Censitários - Alfabetização):

- Contém o quantitativo de pessoas alfabetizadas e não alfabetizadas por faixas etárias, insumo essencial para a construção do indicador de taxa de analfabetismo (15 anos ou mais).

3. dom1.csv (Agregados por Setores Censitários - Domicílios 1):

- Traz a quantidade de domicílios particulares permanentes ocupados e a contagem total de moradores por setor censitário.

4. dom2.csv (Agregados por Setores Censitários - Domicílios 2):

- Fornece os dados de infraestrutura domiciliar urbana: ligação à rede geral de abastecimento de água, esgotamento sanitário (rede geral ou pluvial), cobertura de coleta de lixo e número de banheiros existentes no domicílio.

5. renda.csv (Agregados por Setores Censitários - Renda):

- Contém as métricas de rendimento nominal médio mensal das pessoas responsáveis pelos domicílios com rendimento.

6. Dicionários de Metadados (dicionario_agregados.csv e dicionario_rendimentos.csv):

- Contêm as descrições literais, conceitos e o mapeamento de cada código de variável (por exemplo, V00111, V00309, V06004), assegurando a correta catalogação e interpretação estatística dos campos ao longo do pipeline.

OBS: O IBGE DISPONIBILIZA ESSES DADOS COM VARIÁVEIS NOMINADAS POR CÓDIGOS (CORRESPONDENTES A UMA DESCRIÇÃO NOS DICIONÁRIOS DE DADOS). ESSAS VARIÁVEIS FORAM LISTADAS NOS TÓPICOS NUMÉRICOS IMEDIATAMENTE ANTERIORES E ESTE TEXTO DE OBSERVAÇÃO, MAS O SEU NOME ORIGINAL SERÁ LISTADO QUANDO EXPOSTOS OS MÉTODOS APLICADOS NA ETAPA DE “MODELAGEM E CATÁLOGO DE DADOS”.

1.2.2. Licença de Uso

- **Período de Referência:** Ano de 2022 (coleta do Censo Demográfico 2022 realizada pelo IBGE, refletindo a data-base de referência da operação censitária).
- **Licença de Uso:** Os dados disponibilizados pelo IBGE são de **domínio público institucional / dados abertos governamentais**, regidos pela Política de Dados Abertos do Poder Executivo Federal (Decreto nº 8.777/2016) e pela Lei de Acesso à Informação (Lei nº 12.527/2011). É permitido o uso livre, reprodução, transformação e compartilhamento dos dados, tanto para fins acadêmicos quanto analíticos ou

comerciais, com a exigência formal de atribuição de autoria e citação da fonte
("Fonte: IBGE - Censo Demográfico 2022")

1.3. Perguntas de Negócio

Como instrumento de diagnóstico social do município, o IDS orienta a aplicação do princípio da equidade no planejamento governamental: territórios com menores indicadores demandam alocação prioritária e concentrada de recursos para reduzir disparidades socioespaciais. Por esse motivo, as perguntas de negócio priorizam as favelas e comunidades urbanas de menor pontuação, com o fim de orientar investimentos para intervenções públicas direcionadas onde a vulnerabilidade é mais acentuada.

1. Quais são os 10 bairros com os maiores Índices de Desenvolvimento Social (IDS) em Belo Horizonte e a respectiva renda deles de acordo com o indicador 6 não-normalizado? Com base nessa tabela, analise a homogeneidade ou heterogeneidade desses bairros de acordo com a renda média dos bairros.
2. Quais são os 10 bairros com os menores Índices de Desenvolvimento Social (IDS) em Belo Horizonte e a respectiva renda deles de acordo com o indicador 6 não-normalizado?
3. Quão percentualmente maior ou menor é o IDS mediano das favelas ou comunidades urbanas em relação ao IDS municipal?
4. Alocando o IDS das favelas ou comunidades urbanas em faixas categóricas, analise a concentração percentual dos registros nessas faixas.
5. Dentre a distribuição do IDS das favelas ou comunidades urbanas, quais estão abaixo do limite inferior?
6. Existem setores censitários em situação de vulnerabilidade crítica ($IDS < 0,50$) situados dentro dos 10 bairros de maior IDS?

7. Considerando as mesmas faixas categóricas de IDS, qual é a distribuição percentual dos setores censitários situados na malha urbana regular em comparação com os de favelas e comunidades urbanas?

Todas essas perguntas serão respondidas na etapa “Análise de Dados” e serão mencionadas novamente, na medida em que foram sendo respondidas.

2. Carga dos Dados

Conforme previsto nas orientações deste projeto, o processo de transferência dos dados para o ambiente do Databricks deu-se da seguinte forma:

1. Os arquivos compactados originais foram descarregados diretamente do portal oficial do IBGE para a máquina local;



Figura 1 - Screenshot da página de Downloads dos resultados do Universo agregados por setores censitários do Censo Demográfico de 2022

2. Os arquivos em formato .csv e seus respectivos dicionários foram descompactados e enviados diretamente para o **Unity Catalog Volumes** do Databricks, de nome “IDS_MVP”;
3. O diretório de armazenamento na nuvem padronizado para receber essas fontes brutas foi: “/Volumes/IDS_MVP/bronze/dados_censo_2022/”

Utilizando-se da abordagem da arquitetura medalhão, criou-se um catálogo de nome “IDS_MVP” e, nele, foram criados os *schemas* bronze, silver e gold, de maneira

tal que, cada uma dessas camadas representem um processo de refino, desde a ingestão dos dados brutos (camada Bronze), passando pela limpeza e tratamento de dados (Silver) até chegar à etapa de modelagem (Gold).

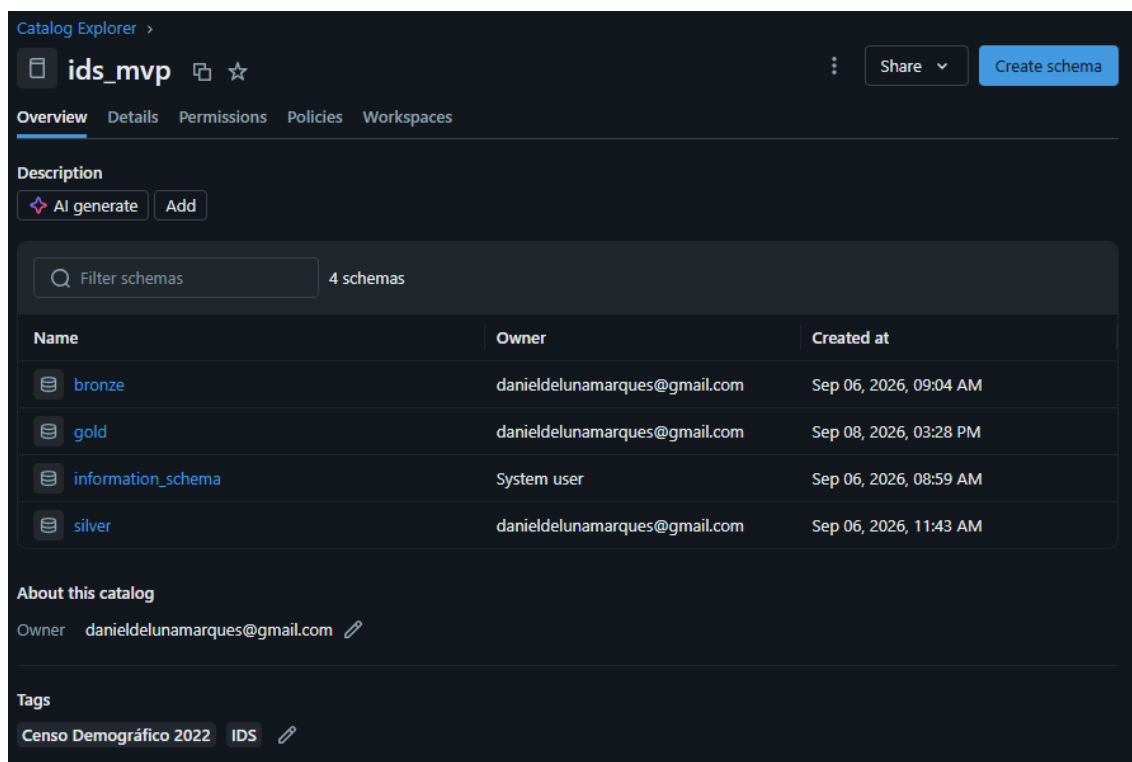
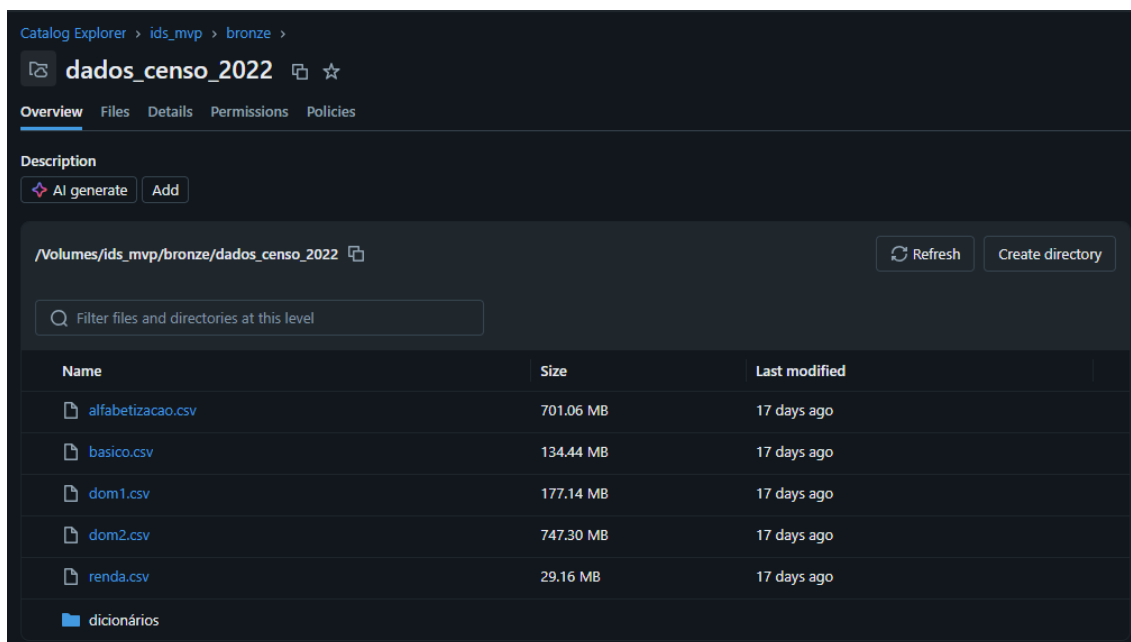


Figura 2 – Overview do Catalog Explorer: IDS_MVP e os schemas criados para as camadas Bronze, Silver e Gold.



Catalog Explorer > ids_mvp > bronze >

dados_censo_2022 ☆

Overview Files Details Permissions Policies

Description

AI generate Add

/Volumes/ids_mvp/bronze/dados_censo_2022 Refresh Create directory

Filter files and directories at this level

Name	Size	Last modified
alfabetizacao.csv	701.06 MB	17 days ago
basico.csv	134.44 MB	17 days ago
dom1.csv	177.14 MB	17 days ago
dom2.csv	747.30 MB	17 days ago
renda.csv	29.16 MB	17 days ago
dicionários		

Figura 3 - Overview do Volumes na camada bronze, com os dados ingeridos em sua forma bruta.

A partir dessa estrutura persistida no Volume, a etapa seguinte realiza a ingestão distribuída via PySpark, aplicando a codificação e os delimitadores pertinentes para a criação das tabelas Delta brutas na camada Bronze.

A partir de agora, serão mostrados os *screenshots* da primeira etapa do processo realizado em *PySpark*: a etapa de ingestão dos dados a partir dos arquivos existentes no Volume do esquema da camada Bronze.

```

# Vou padronizar a leitura de todos os arquivos usando ponto e vírgula e encoding latini por conta da formatação do IBGE.
# As tabelas serão salvas no formato Delta dentro do esquema bronze para garantir o histórico bruto.

# 1. Arquivo Básico
df_basico = spark.read.format("csv") \
    .option("header", "true") \
    .option("sep", ";") \
    .option("inferSchema", "true") \
    .option("encoding", "latin1") \
    .load("/Volumes/IDS_MVP/bronze/dados_censo_2022/basico.csv")

df_basico.write.format("delta").mode("overwrite").saveAsTable("IDS_MVP.bronze.basico")

# 2. Arquivo de Alfabetização
df_alfabetizacao = spark.read.format("csv") \
    .option("header", "true") \
    .option("sep", ";") \
    .option("inferSchema", "true") \
    .option("encoding", "utf8") \
    .load("/Volumes/IDS_MVP/bronze/dados_censo_2022/alfabetizacao.csv")

df_alfabetizacao.write.format("delta").mode("overwrite").saveAsTable("IDS_MVP.bronze.alfabetizacao")

# 3. Arquivo Domicílios 1
df_dom1 = spark.read.format("csv") \
    .option("header", "true") \
    .option("sep", ";") \
    .option("inferSchema", "true") \
    .option("encoding", "utf8") \
    .load("/Volumes/IDS_MVP/bronze/dados_censo_2022/dom1.csv")

df_dom1.write.format("delta").mode("overwrite").saveAsTable("IDS_MVP.bronze.dom1")

# 4. Arquivo Domicílios 2
df_dom2 = spark.read.format("csv") \
    .option("header", "true") \
    .option("sep", ";") \
    .load("/Volumes/IDS_MVP/bronze/dados_censo_2022/dom2.csv")

df_dom2.write.format("delta").mode("overwrite").saveAsTable("IDS_MVP.bronze.dom2")
```

Figura 4- Parte 1 do código de ingestão de dados a partir da camada Bronze e a criação das tabelas deltas nessa mesma camada.

```

.option("sep", ";") \
    .option("inferSchema", "true") \
    .option("encoding", "utf8") \
    .load("/Volumes/IDS_MVP/bronze/dados_censo_2022/renda.csv")

df_renda.write.format("delta").mode("overwrite").saveAsTable("IDS_MVP.bronze.renda")

# Ingestão do Dicionário Agregados
df_dic_basico = spark.read.format("csv") \
    .option("header", "true") \
    .option("sep", ";") \
    .option("inferSchema", "true") \
    .option("encoding", "utf8") \
    .load("/Volumes/ids_mvp/bronze/dados_censo_2022/dicionários/dicionario_agregados.csv")

df_dic_basico.write.format("delta").mode("overwrite").saveAsTable("IDS_MVP.bronze.dicionario_agregados")

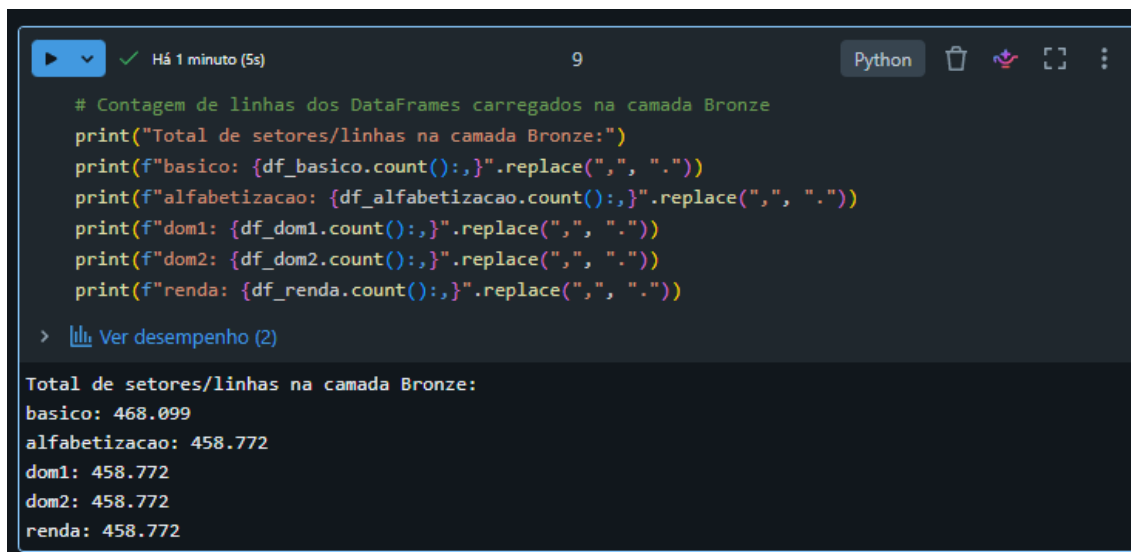
# Ingestão do Dicionário Renda
df_dic_basico = spark.read.format("csv") \
    .option("header", "true") \
    .option("sep", ";") \
    .option("inferSchema", "true") \
    .option("encoding", "utf8") \
    .load("/Volumes/ids_mvp/bronze/dados_censo_2022/dicionários/dicionario_rendimentos.csv")

df_dic_basico.write.format("delta").mode("overwrite").saveAsTable("IDS_MVP.bronze.dicionario_renda")

> See performance \(7\) ✓ Checked for improvements

6 dataframes: df_alfabetizacao, df_basico, df_dic_basico, df_dom1, df_dom2, df_re...
> df_alfabetizacao: pyspark.sql.connect.dataframe.DataFrame = [CD_setor: long, V00644: string ... 361 more fields]
> df_basico: pyspark.sql.connect.dataframe.DataFrame = [CD_SETOR: long, SITUACAO: string ... 36 more fields]
> df_dic_basico: pyspark.sql.connect.dataframe.DataFrame = [Tema: string, Variável: string ... 1 more field]
> df_dom1: pyspark.sql.connect.dataframe.DataFrame = [CD_setor: long, V00001: string ... 88 more fields]
> df_dom2: pyspark.sql.connect.dataframe.DataFrame = [setor: long, V00090: string ... 405 more fields]
> df_renda: pyspark.sql.connect.dataframe.DataFrame = [CD_SETOR: long, V06001: string ... 5 more fields]
```

Figura 5 - Parte 2 do código de ingestão de dados a partir da camada Bronze e a criação das tabelas delta nessa mesma camada.



```
# Contagem de linhas dos DataFrames carregados na camada Bronze
print("Total de setores/linhas na camada Bronze:")
print(f"basico: {df_basico.count():,}".replace(",", "."))
print(f"alfabetizacao: {df_alfabetizacao.count():,}".replace(",", "."))
print(f"dom1: {df_dom1.count():,}".replace(",", "."))
print(f"dom2: {df_dom2.count():,}".replace(",", "."))
print(f"renda: {df_renda.count():,}".replace(",", "."))
```

> [Ver desempenho \(2\)](#)

```
Total de setores/linhas na camada Bronze:
basico: 468.099
alfabetizacao: 458.772
dom1: 458.772
dom2: 458.772
renda: 458.772
```

Figura 6 - Linhas dos arquivos do Censo Demográfico de 2022 que foram ingeridos.

Esses dados brutos, na forma como foram ingeridos, possuem todos os setores censitários do Brasil. No decorrer deste notebook será mencionado novamente a seguinte sutileza: o arquivo "básico" possui um número maior de setores censitários (e, portanto, de linhas), pois os demais arquivos (alfabetizacao, renda, dom1 e dom2) não possuem os setores vazios (sem domicílios).

A próxima etapa será a Qualidade de Dados, que, em razão da organização imanente à arquitetura medalhão, seguida neste projeto, consistirá em todo o processo de limpeza e tratamento realizado na camada Silver.

3. Qualidade de Dados

Na etapa de Qualidade de Dados, a primeira validação referiu-se à completude e à consistência da malha censitária no processo inicial de limpeza. Inicialmente, verificou-se que duas das cinco tabelas, Alfabetização e Domicílios 2¹ não possuíam o mesmo nome para a chave de ligação entre elas, isto é, a variável CD_SETOR (Código do Setor Censitário): a primeira continha “CD_setor” e a segunda “setor” como nome, respectivamente. Então, renomeou-se as variáveis:

```
from pyspark.sql.functions import col

# lendo as tabelas brutas que ja foram ingeridas na bronze
df_basico = spark.table("IDS_MVP.bronze.basico")
df_alfabetizacao = spark.table("IDS_MVP.bronze.alfabetizacao")
df_dom1 = spark.table("IDS_MVP.bronze.dom1")
df_dom2 = spark.table("IDS_MVP.bronze.dom2")
df_renda = spark.table("IDS_MVP.bronze.renda")

# primeiro passo: padronizar a chave do setor
df_alfabetizacao = df_alfabetizacao.withColumnRenamed("CD_setor", "CD_SETOR")
df_dom2 = df_dom2.withColumnRenamed("setor", "CD_SETOR")
```

Figura 7 - Código de renomeação de variáveis das tabelas alfabetizacao e dom2, oriundas da camada Bronze

Feito isso, procedeu-se à criação da tabela com a relação de setores censitários, bairros e favelas, para o município de Belo Horizonte, capital do estado de Minas Gerais, objeto de estudo deste projeto.

¹ No notebook que possui o pipeline, essas tabelas tiveram o nome abreviado para facilitar o processo de programação PySpark. São elas: Básico (basico), Alfabetização (alfabetizacao), Domicílios 1 (dom1) e Domicílios 2 (dom2).

```
▶ ✓ Sep 16, 2026 (5s) 13 Python [ ] [ ] [ ]

# o basico tem o municipio e a uf de cada setor
# filtrando apenas Belo Horizonte / Minas Gerais
relacao_setor_bairro = (
    df_basico
    .filter(col("NM_UF") == "Minas Gerais")
    .filter(col("NM_MUN") == "Belo Horizonte")
    .select("CD_SETOR", "NM_UF", "NM_MUN", "NM_BAIRRO", "NM_FCU")
    .dropDuplicates(["CD_SETOR"])
)

# relacao setor-bairro e favelas do municipio
# fica de referencia para os joins e para filtrar as outras tabelas
relacao_setor_bairro.write.format("delta").mode("overwrite").saveAsTable("IDS_MVP.silver.relacao_setor_bairro")

# print de checagem no recorte ou subdivisao de setores censitarios
print("setores em BH:", relacao_setor_bairro.count())
relacao_setor_bairro.show(10, truncate=False)

> See performance \(3\) ✓ Checked for improvements

> relacao_setor_bairro: pyspark.sql.connect.dataframe.DataFrame = [CD_SETOR: long, NM_UF: string ... 3 more fields]

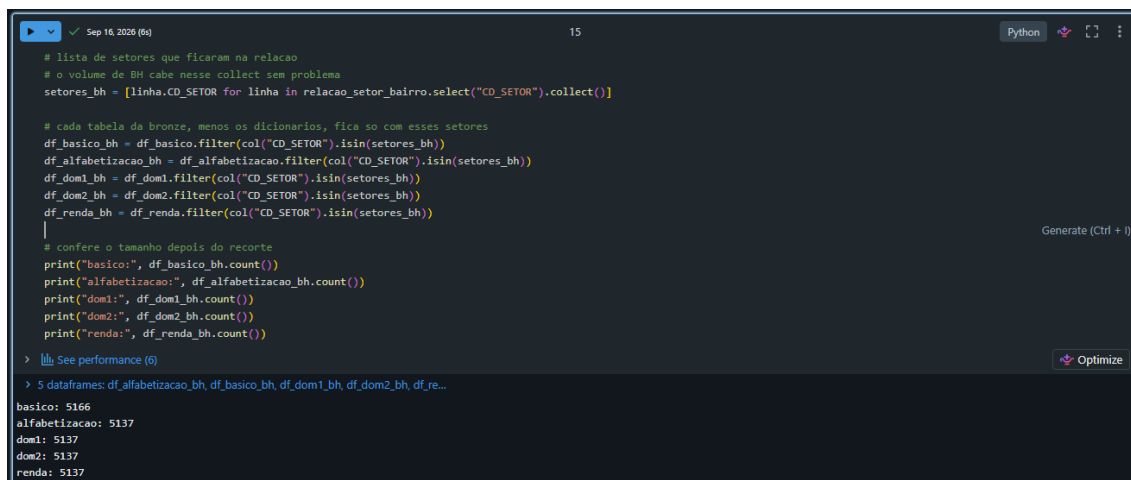
setores em BH: 5166

+-----+-----+-----+-----+-----+
|CD_SETOR|NM_UF|NM_MUN|NM_BAIRRO|NM_FCU|
+-----+-----+-----+-----+-----+
|310620005620016|Minas Gerais|Belo Horizonte|Funcionários|NULL|
|310620005620017|Minas Gerais|Belo Horizonte|Funcionários|NULL|
|310620005620041|Minas Gerais|Belo Horizonte|Carmo|NULL|
|310620005620052|Minas Gerais|Belo Horizonte|Santa Efigênia|NULL|
|310620005620091|Minas Gerais|Belo Horizonte|Boa Viagem|NULL|
|310620005620222|Minas Gerais|Belo Horizonte|Sion|NULL|
|310620005620229|Minas Gerais|Belo Horizonte|Sion|NULL|
|310620005620262|Minas Gerais|Belo Horizonte|Sion|NULL|
|310620005620293|Minas Gerais|Belo Horizonte|Vila Santa Lúcia|Vila Barragem Santa Lucia|
|310620005620308|Minas Gerais|Belo Horizonte|Santo Antônio|NULL|
+-----+-----+-----+-----+-----+
only showing top 10 rows
```

Figura 8 - Código de criação da tabela setor-bairro-favela, a partir filtragem da variável NM_MUN para o município de Belo Horizonte.

Como é possível verificar na imagem acima, nem todos os setores censitários apresentam informação para a variável NM_FCU (nome da favela ou comunidade urbana), pois, nem todos os setores censitários estão inseridos em território de favelas. Então, esses valores nulos são esperados.

A tabela de relação setor-bairro foi utilizada como referência para a filtragem dos outros conjuntos de dados, isto é, aquelas das temáticas de Alfabetização, Renda, Domicílios 1, Domicílios 2 e a Básica:



```
# lista de setores que ficaram na relacao
# o volume de BH cabe nesse collect sem problema
setores_bh = [linha.CD_SETOR for linha in relacao_setor_bairro.select("CD_SETOR").collect()]

# cada tabela da bronze, menos os dicionarios, fica so com esses setores
df_basico_bh = df_basico.filter(col("CD_SETOR").isin(setores_bh))
df_alfabetizacao_bh = df_alfabetizacao.filter(col("CD_SETOR").isin(setores_bh))
df_dom1_bh = df_dom1.filter(col("CD_SETOR").isin(setores_bh))
df_dom2_bh = df_dom2.filter(col("CD_SETOR").isin(setores_bh))
df_renda_bh = df_renda.filter(col("CD_SETOR").isin(setores_bh))

# confere o tamanho depois do recorte
print("basico:", df_basico_bh.count())
print("alfabetizacao:", df_alfabetizacao_bh.count())
print("dom1:", df_dom1_bh.count())
print("dom2:", df_dom2_bh.count())
print("renda:", df_renda_bh.count())

> 5 dataframes: df_alfabetizacao_bh, df_basico_bh, df_dom1_bh, df_dom2_bh, df_re...
```

basico: 5166
alfabetizacao: 5137
dom1: 5137
dom2: 5137
renda: 5137

Figura 9- Filtrando os setores censitários as tabelas temáticas a partir da relação setor-bairro do município de Belo Horizonte.

Ao realizar a filtragem territorial exclusiva para o município de Belo Horizonte na camada Silver, constatou-se uma divergência natural no quantitativo de setores censitários: enquanto o arquivo de malha cadastral básica registrava 5.166 setores, os arquivos temáticos (renda, alfabetização e domicílios) apresentavam apenas 5.137 setores. Essa redução, contudo, não se trata de uma falha de integridade ou perda de dados, mas sim de uma adequação metodológica do próprio IBGE, que omite agregados de setores sem moradores dos arquivos temáticos, atestando a consistência da base de dados fornecida para a análise. Essa informação pode ser encontrada [clikando aqui](#).

O IDS possui uma estrutura bem definida de indicadores que são baseados em algumas variáveis do Censo 2022, mas não em todas. Esses indicadores -- que serão produzidos na camada Gold -- possuem por insumos as variáveis listadas abaixo. Por isso, cada uma das tabelas passou por uma seleção de variáveis:

Indicador 1 (Domicílios 1):

- V00111: Domicílios Particulares Permanentes Ocupados, Utiliza rede geral de distribuição
- V00001: Domicílios Particulares Permanentes Ocupados

Indicador 2:

- V00309: Domicílios Particulares Permanentes Ocupados, Destinação do esgoto do banheiro ou sanitário ou buraco para dejeções é rede geral ou pluvial
- V00001: Domicílios Particulares Permanentes Ocupados

Indicador 3:

- V00397: Domicílios Particulares Permanentes Ocupados, Lixo coletado no domicílio por serviço de limpeza
- V00398: Domicílios Particulares Permanentes Ocupados, Lixo depositado em caçamba de serviço de limpeza
- V00001: Domicílios Particulares Permanentes Ocupados

Indicador 4:

- V00485: Domicílios Particulares Permanentes Ocupados, 1 banheiro de uso exclusivo com chuveiro e vaso sanitário existentes no domicílio
- V00486: Domicílios Particulares Permanentes Ocupados, 2 banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicílio
- V00487: Domicílios Particulares Permanentes Ocupados, 3 banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicílio
- V00488: Domicílios Particulares Permanentes Ocupados, 4 banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicílio
- V00489: Domicílios Particulares Permanentes Ocupados, 5 banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicílio
- V00490: Domicílios Particulares Permanentes Ocupados, 6 banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicílio
- V00491: Domicílios Particulares Permanentes Ocupados, 7 banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicílio
- V00492: Domicílios Particulares Permanentes Ocupados, 8 banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicílio

- V00493: Domicílios Particulares Permanentes Ocupados, 9 ou mais banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicílio
- V00005: Domicílios Particulares Permanentes Ocupados, Quantidade de moradores

Indicador 5:

- V00853: 15 a 29 anos, Morador não sabe ler e escrever
- V00855: 30 a 59 anos, Morador não sabe ler e escrever
- V00857: 60 anos ou mais, Morador não sabe ler e escrever

Indicador 6:

- V06004: Valor do rendimento nominal médio mensal das pessoas responsáveis com rendimentos por domicílios particulares permanentes ocupados

```
# Sep 16, 2024 (Su) 10

# colunas que cada tabela precisa carregar daqui pra frente
colunas_dom1 = ["CD_SETOR", "V00001", "V00005"]
colunas_dom2 = ["CD_SETOR", "V00111", "V00309", "V00397", "V00398",
                "V00485", "V00486", "V00487", "V00488", "V00489",
                "V00490", "V00491", "V00492", "V00493"]
colunas_alfabetizacao = ["CD_SETOR", "V00852", "V00853", "V00854", "V00855", "V00856", "V00857"]
colunas_basico = ["CD_SETOR", "NM_FCU"]
colunas_renda = ["CD_SETOR", "V06004"]

# recorta cada df ja filtrado de bh
df_dom1_bh = df_dom1.select(colunas_dom1)
df_dom2_bh = df_dom2.select(colunas_dom2)
df_alfabetizacao_bh = df_alfabetizacao.select(colunas_alfabetizacao)
df_basico_bh = df_basico.select(colunas_basico)
df_renda_bh = df_renda.select(colunas_renda)

# confere o que ficou
print("dom1:", df_dom1_bh.columns)
print("dom2:", df_dom2_bh.columns)
print("alfabetizacao:", df_alfabetizacao_bh.columns)
print("basico:", df_basico_bh.columns)
print("renda:", df_renda_bh.columns)

> 5 dataframes: df_alfabetizacao_bh, df_basico_bh, df_dom1_bh, df_dom2_bh, df_renda_bh
dom1: ["CD_SETOR", "V00001", "V00005"]
dom2: ["CD_SETOR", "V00111", "V00309", "V00397", "V00398", "V00485", "V00486", "V00487", "V00488", "V00489", "V00490", "V00491", "V00492", "V00493"]
alfabetizacao: ["CD_SETOR", "V00852", "V00853", "V00854", "V00855", "V00856", "V00857"]
basico: ["CD_SETOR", "NM_FCU"]
renda: ["CD_SETOR", "V06004"]
```

Figura 10 - Seleção de variáveis nas tabelas

Após essa seleção, foi verificado a tipagem dos dados: com exceção das variáveis que possuíam caráter de recorte geográfico (nome de município, bairro, favela e o próprio código do setor censitário), todas as demais estar com a tipagem de inteiro (*int*) e, no caso da variável de renda (V06004) como contínua (*float*). Essa verificação encontra-se abaixo:

```
print("==== schema dom1 ====")
df_dom1_bh.printSchema()
print(df_dom1_bh.dtypes)

print("==== schema dom2 ====")
df_dom2_bh.printSchema()
print(df_dom2_bh.dtypes)

print("==== schema alfabetizacao ====")
df_alfabetizacao_bh.printSchema()
print(df_alfabetizacao_bh.dtypes)

print("==== schema renda ====")
df_renda_bh.printSchema()
print(df_renda_bh.dtypes)

print("==== schema basico ====")
df_basico_bh.printSchema()
print(df_basico_bh.dtypes)

==== schema dom1 ====
root
|-- CD_SETOR: long (nullable = true)
|-- V00001: string (nullable = true)
|-- V00005: string (nullable = true)

[('CD_SETOR', 'bigint'), ('V00001', 'string'), ('V00005', 'string')]
==== schema dom2 ====
root
|-- CD_SETOR: long (nullable = true)
|-- V00011: string (nullable = true)
|-- V00009: string (nullable = true)
|-- V00037: string (nullable = true)
|-- V00038: string (nullable = true)
|-- V00045: string (nullable = true)
|-- V00046: string (nullable = true)
|-- V00047: string (nullable = true)
|-- V00048: string (nullable = true)
|-- V00049: string (nullable = true)
|-- V00049: string (nullable = true)
|-- V00049: string (nullable = true)
```

Figura 11 - Verificação da tipagem dos dados

Todos eles estavam como string, com exceção de CD_SETOR, que estava como long (é um tipo de dado numérico inteiro de 64 bits com sinal. Para mais detalhes, acesso este link: [Tipos de dados PySpark](#)). Então, foi preciso verificar se existiam valores não numéricos nessas variáveis:

```
from pyspark.sql.functions import col, trim

# CD_SETOR fica de fora (chave)
# NM_FCU fica de fora (nome de favela)

# inteiro: sinal opcional + so digitos
padrao_int = "^[0-9]+$"

# float de renda: ponto ou virgula no decimal
padrao_float = "^[0-9]+([.][0-9])?$"

# dom1
# ----- DOM1 -----
print("===== DOM1 =====")

# V000001: domicilios particulares permanentes ocupados
v = {
    df_dom1_bh
        .select(trim(col("V000001")).cast("string").alias("valor"))
        .filter(col("valor").isNotNull())
        .filter(col("valor") != "")
        .filter(~col("valor").rlike(padrao_int))
        .distinct()
}
print("V000001 - valores nao numericos:", v.count())
v.show(100, truncate=False)

# V000005: quantidade de moradores
v = {
    df_dom1_bh
        .select(trim(col("V000005")).cast("string").alias("valor"))
        .filter(col("valor").isNotNull())
        .filter(col("valor") != "")
        .filter(~col("valor").rlike(padrao_int))
        .distinct()
}
print("V000005 - valores nao numericos:", v.count())
v.show(100, truncate=False)

# dom2
# ----- DOM2 -----
print("===== DOM2 =====")

# V000111: usa rede geral de distribuicao de agua
```

Figura 12 - Parte 1 Script de Identificação de Valores Não Numéricos

```
v = {
    df_dom2_bh
        .select(trim(col("V000111")).cast("string").alias("valor"))
        .filter(col("valor").isNotNull())
        .filter(col("valor") != "")
        .filter(~col("valor").rlike(padrao_int))
        .distinct()
}
print("V000111 - valores nao numericos:", v.count())
v.show(100, truncate=False)

# V000309: esgoto na rede geral ou pluvial
v = {
    df_dom2_bh
        .select(trim(col("V000309")).cast("string").alias("valor"))
        .filter(col("valor").isNotNull())
        .filter(col("valor") != "")
        .filter(~col("valor").rlike(padrao_int))
        .distinct()
}
print("V000309 - valores nao numericos:", v.count())
v.show(100, truncate=False)

# V000397: lixo coletado no domicilio
v = {
    df_dom2_bh
        .select(trim(col("V000397")).cast("string").alias("valor"))
        .filter(col("valor").isNotNull())
        .filter(col("valor") != "")
        .filter(~col("valor").rlike(padrao_int))
        .distinct()
}
print("V000397 - valores nao numericos:", v.count())
v.show(100, truncate=False)

# V000398: lixo em cacamba de servico da limpeza
v = {
    df_dom2_bh
        .select(trim(col("V000398")).cast("string").alias("valor"))
        .filter(col("valor").isNotNull())
        .filter(col("valor") != "")
        .filter(~col("valor").rlike(padrao_int))
        .distinct()
}
print("V000398 - valores nao numericos:", v.count())
v.show(100, truncate=False)
```

Figura 13 - Parte 2 Script de Identificação de Valores Não Numéricos

```
# V00485: 1 banheiros
v = {
  df_dom2_bh
  .select(trim(col("V00485")).cast("string")).alias("valor")
  .filter(col("valor").isNotNull())
  .filter(col("valor") != "")
  .filter(~col("valor").rlike(padrao_int))
  .distinct()
}
print("V00485 - valores nao numericos:", v.count())
v.show(100, truncate=false)

# V00486: 2 banheiros
v = {
  df_dom2_bh
  .select(trim(col("V00486")).cast("string")).alias("valor")
  .filter(col("valor").isNotNull())
  .filter(col("valor") != "")
  .filter(~col("valor").rlike(padrao_int))
  .distinct()
}
print("V00486 - valores nao numericos:", v.count())
v.show(100, truncate=false)

# V00487: 3 banheiros
v = {
  df_dom2_bh
  .select(trim(col("V00487")).cast("string")).alias("valor")
  .filter(col("valor").isNotNull())
  .filter(col("valor") != "")
  .filter(~col("valor").rlike(padrao_int))
  .distinct()
}
print("V00487 - valores nao numericos:", v.count())
v.show(100, truncate=false)

# V00488: 4 banheiros
v = {
  df_dom2_bh
  .select(trim(col("V00488")).cast("string")).alias("valor")
  .filter(col("valor").isNotNull())
  .filter(col("valor") != "")
  .filter(~col("valor").rlike(padrao_int))
  .distinct()
}
print("V00488 - valores nao numericos:", v.count())
v.show(100, truncate=false)
```

Figura 14 - Parte 3 Script de Identificação de Valores Não Numéricos

```
# V00489: 5 banheiros
v = {
  df_dom2_bh
  .select(trim(col("V00489")).cast("string")).alias("valor")
  .filter(col("valor").isNotNull())
  .filter(col("valor") != "")
  .filter(~col("valor").rlike(padrao_int))
  .distinct()
}
print("V00489 - valores nao numericos:", v.count())
v.show(100, truncate=false)

# V00490: 6 banheiros
v = {
  df_dom2_bh
  .select(trim(col("V00490")).cast("string")).alias("valor")
  .filter(col("valor").isNotNull())
  .filter(col("valor") != "")
  .filter(~col("valor").rlike(padrao_int))
  .distinct()
}
print("V00490 - valores nao numericos:", v.count())
v.show(100, truncate=false)

# V00491: 7 banheiros
v = {
  df_dom2_bh
  .select(trim(col("V00491")).cast("string")).alias("valor")
  .filter(col("valor").isNotNull())
  .filter(col("valor") != "")
  .filter(~col("valor").rlike(padrao_int))
  .distinct()
}
print("V00491 - valores nao numericos:", v.count())
v.show(100, truncate=false)

# V00492: 8 banheiros
v = {
  df_dom2_bh
  .select(trim(col("V00492")).cast("string")).alias("valor")
  .filter(col("valor").isNotNull())
  .filter(col("valor") != "")
  .filter(~col("valor").rlike(padrao_int))
  .distinct()
}
print("V00492 - valores nao numericos:", v.count())
v.show(100, truncate=false)
```

Figura 15 - Parte 4 Script de Identificação de Valores Não Numéricos

```
# V00853: 15 a 20 anos, sabe ler
v = {
  df_don1_bh
  .select(trim(col("V00853")).cast("string").alias("valor"))
  .filter(col("valor").isNotNull())
  .filter(col("valor") != "")
  .filter(col("valor").rlike(padrao_int))
  .distinct()
}
print("V00853 - valores nao numericos:", v.count())
v.show(100, truncate=False)

# alfabetizacao
# ----- ALFABETIZACAO -----
# V00852: 15 a 20 anos, sabe ler
v = {
  df_alfabetizacao_bh
  .select(trim(col("V00852")).cast("string").alias("valor"))
  .filter(col("valor").isNotNull())
  .filter(col("valor") != "")
  .filter(col("valor").rlike(padrao_int))
  .distinct()
}
print("V00852 - valores nao numericos:", v.count())
v.show(100, truncate=False)

# V00853: 15 a 20 anos, nao sabe ler
v = {
  df_alfabetizacao_bh
  .select(trim(col("V00853")).cast("string").alias("valor"))
  .filter(col("valor").isNotNull())
  .filter(col("valor") != "")
  .filter(col("valor").rlike(padrao_int))
  .distinct()
}
print("V00853 - valores nao numericos:", v.count())
v.show(100, truncate=False)

# V00854: 20 a 50 anos, sabe ler
v = {
  df_alfabetizacao_bh
  .select(trim(col("V00854")).cast("string").alias("valor"))
  .filter(col("valor").isNotNull())
  .filter(col("valor") != "")
  .filter(col("valor").rlike(padrao_int))
  .distinct()
}
print("V00854 - valores nao numericos:", v.count())
v.show(100, truncate=False)
```

Figura 16 - Parte 5 Script de Identificação de Valores Não Numéricos

```
# V00856: 60 anos ou mais, sabe ler
v = {
  df_alfabetizacao_bh
  .select(trim(col("V00856")).cast("string").alias("valor"))
  .filter(col("valor").isNotNull())
  .filter(col("valor") != "")
  .filter(col("valor").rlike(padrao_int))
  .distinct()
}
print("V00856 - valores nao numericos:", v.count())
v.show(100, truncate=False)

# V00857: 60 anos ou mais, nao sabe ler
v = {
  df_alfabetizacao_bh
  .select(trim(col("V00857")).cast("string").alias("valor"))
  .filter(col("valor").isNotNull())
  .filter(col("valor") != "")
  .filter(col("valor").rlike(padrao_int))
  .distinct()
}
print("V00857 - valores nao numericos:", v.count())
v.show(100, truncate=False)

# renda
# ----- RENDA -----
# V00804: rendimento medio do responsavel
v = {
  df_renda_bh
  .select(trim(col("V00804")).cast("string").alias("valor"))
  .filter(col("valor").isNotNull())
  .filter(col("valor") != "")
  .filter(col("valor").rlike(padrao_float))
  .distinct()
}
print("V00804 - valores nao numericos:", v.count())
v.show(100, truncate=False)
```

Figura 17- Parte 6 Script de Identificação de Valores Não Numéricos

As figuras acima mostram *screenshots* do código; a figura abaixo mostra uma amostra dos resultados:

```
> See performance (44) ✓ Checked for improvements
> v: pyspark.sql.connect.dataframe.DataFrame = [valor: string]

===== DOM1 =====
V00001 - valores nao numericos: 1
+-----+
|valor|
+-----+
|X |
+-----+

V00005 - valores nao numericos: 1
+-----+
|valor|
+-----+
|X |
+-----+

===== DOM2 =====
V00111 - valores nao numericos: 1
+-----+
|valor|
+-----+
|X |
```

Figura 18 - Resultados da Identificação de valores Não Numéricos

A razão para que todas as variáveis estejam sendo tipadas como *string* é a existência de valores do tipo "X". [De acordo com os documentos metodológicos oficiais do IBGE para o Censo Demográfico 2022](#), os valores que aparecem preenchidos com a letra "X" nos arquivos de agregados por setores censitários indicam a omissão de dados por motivos de sigilo estatístico e proteção à privacidade dos informantes.

Com isso, foi preciso remover todos os valores X, substituindo-os por *null* e passar todas as variáveis para *int*, com exceção da V06004, que deveria ser *float*.

```
from pyspark.sql.functions import col, when, trim, regexp_replace

# X (com ou sem espaço na borda) vira nulo
# o restante permanece como esta ate o cast

# dom1
# -----
df_dom1_bh = (
    df_dom1_bh
    .withColumn("V00001", when(trim(col("V00001")) == "X", None).otherwise(col("V00001")))
    .withColumn("V00005", when(trim(col("V00005")) == "X", None).otherwise(col("V00005")))
    .withColumn("V00001", col("V00001").cast("int"))
    .withColumn("V00005", col("V00005").cast("int"))
)

# dom2
# -----
df_dom2_bh = (
    df_dom2_bh
    .withColumn("V00111", when(trim(col("V00111")) == "X", None).otherwise(col("V00111")))
    .withColumn("V00309", when(trim(col("V00309")) == "X", None).otherwise(col("V00309")))
    .withColumn("V00397", when(trim(col("V00397")) == "X", None).otherwise(col("V00397")))
    .withColumn("V00398", when(trim(col("V00398")) == "X", None).otherwise(col("V00398")))
    .withColumn("V00485", when(trim(col("V00485")) == "X", None).otherwise(col("V00485")))
    .withColumn("V00486", when(trim(col("V00486")) == "X", None).otherwise(col("V00486")))
    .withColumn("V00487", when(trim(col("V00487")) == "X", None).otherwise(col("V00487")))
    .withColumn("V00488", when(trim(col("V00488")) == "X", None).otherwise(col("V00488")))
    .withColumn("V00489", when(trim(col("V00489")) == "X", None).otherwise(col("V00489")))
    .withColumn("V00490", when(trim(col("V00490")) == "X", None).otherwise(col("V00490")))
    .withColumn("V00491", when(trim(col("V00491")) == "X", None).otherwise(col("V00491")))
    .withColumn("V00492", when(trim(col("V00492")) == "X", None).otherwise(col("V00492")))
    .withColumn("V00493", when(trim(col("V00493")) == "X", None).otherwise(col("V00493")))
    .withColumn("V00111", col("V00111").cast("int"))
    .withColumn("V00309", col("V00309").cast("int"))
    .withColumn("V00397", col("V00397").cast("int"))
    .withColumn("V00398", col("V00398").cast("int"))
    .withColumn("V00485", col("V00485").cast("int"))
    .withColumn("V00486", col("V00486").cast("int"))
    .withColumn("V00487", col("V00487").cast("int"))
    .withColumn("V00488", col("V00488").cast("int"))
    .withColumn("V00489", col("V00489").cast("int"))
    .withColumn("V00490", col("V00490").cast("int"))
    .withColumn("V00491", col("V00491").cast("int"))
    .withColumn("V00492", col("V00492").cast("int"))
    .withColumn("V00493", col("V00493").cast("int"))
)
```

Figura 19 - Parte 1 do código feito para remover os valores "X" das variáveis e passá-las para int ou float.

```
# alfabetizacao
# -----
df_alfabetizacao_bh = (
    df_alfabetizacao_bh
    .withColumn("V00852", when(trim(col("V00852")) == "X", None).otherwise(col("V00852")))
    .withColumn("V00853", when(trim(col("V00853")) == "X", None).otherwise(col("V00853")))
    .withColumn("V00854", when(trim(col("V00854")) == "X", None).otherwise(col("V00854")))
    .withColumn("V00855", when(trim(col("V00855")) == "X", None).otherwise(col("V00855")))
    .withColumn("V00856", when(trim(col("V00856")) == "X", None).otherwise(col("V00856")))
    .withColumn("V00857", when(trim(col("V00857")) == "X", None).otherwise(col("V00857")))
    .withColumn("V00852", col("V00852").cast("int"))
    .withColumn("V00853", col("V00853").cast("int"))
    .withColumn("V00854", col("V00854").cast("int"))
    .withColumn("V00855", col("V00855").cast("int"))
    .withColumn("V00856", col("V00856").cast("int"))
    .withColumn("V00857", col("V00857").cast("int"))
)

# renda

# tira o X
# se o decimal vier com vírgula, troca por ponto so no cast
df_renda_bh = (
    df_renda_bh
    .withColumn("V06004", when(trim(col("V06004")) == "X", None).otherwise(col("V06004")))
    .withColumn("V06004", regexp_replace(col("V06004"), ",", ".").cast("float"))
)

> 4 dataframes: df_alfabetizacao_bh, df_dom1_bh, df_dom2_bh, df_renda_bh
```

Figura 20 - Parte 2 do código feito para remover os valores "X" das variáveis e passá-las para int ou float.

Feitas essas modificações, os dados foram gravados no *schema* (ou, conceitualmente, na camada) Silver:

```
✓ Sep 16, 2024 (12h) 27 Python
# overwrite: se a tabela ja existir, substitui
# mesmo nome da bronze, agora no schema silver

df_dom1_bh.write.format("delta").mode("overwrite").saveAsTable("IDS_MVP.silver.dom1")
df_dom2_bh.write.format("delta").mode("overwrite").saveAsTable("IDS_MVP.silver.dom2")
df_alfabetizacao_bh.write.format("delta").mode("overwrite").saveAsTable("IDS_MVP.silver.alfabetizacao")
df_renda_bh.write.format("delta").mode("overwrite").saveAsTable("IDS_MVP.silver.renda")

# relacao setor-bairro e favelas, usada como referencia
relacao_setor_bairro.write.format("delta").mode("overwrite").saveAsTable("IDS_MVP.silver.relacao_setor_bairro")

> See performance (5) ✓ Checked for improvements
```

Figura 21- Código de gravação das tabelas na camada Silver

Preenchida a camada Silver com os insumos para cálculo dos indicadores, foi necessário preencher os metadados:

Pós-Graduação em Ciência de Dados e Analytics

Sprint de Engenharia de Dados

Aluno: Daniel de Luna Marques

Matrícula: 4052026000228



```
# relacao_setor_bairro

spark.sql("COMMENT ON TABLE IDS_MVP.silver.relacao_setor_bairro IS 'Relacao entre setor censitario, bairro e favela ou comunidade urbana de Belo Horizonte'")

spark.sql("ALTER TABLE IDS_MVP.silver.relacao_setor_bairro ALTER COLUMN CD_SETOR COMMENT 'Código do Setor Censitário')
spark.sql("ALTER TABLE IDS_MVP.silver.relacao_setor_bairro ALTER COLUMN NM_UP COMMENT 'Unidade Federativa')
spark.sql("ALTER TABLE IDS_MVP.silver.relacao_setor_bairro ALTER COLUMN NM_MUN COMMENT 'Município')
spark.sql("ALTER TABLE IDS_MVP.silver.relacao_setor_bairro ALTER COLUMN NM_BAIRRO COMMENT 'Bairro')
spark.sql("ALTER TABLE IDS_MVP.silver.relacao_setor_bairro ALTER COLUMN NM_FCU COMMENT 'Favela ou Comunidade Urbana'")

spark.sql("ALTER TABLE IDS_MVP.silver.relacao_setor_bairro ALTER COLUMN CD_SETOR SET TAGS ('Censo Demográfico 2022')")
spark.sql("ALTER TABLE IDS_MVP.silver.relacao_setor_bairro ALTER COLUMN NM_UP SET TAGS ('Censo Demográfico 2022')")
spark.sql("ALTER TABLE IDS_MVP.silver.relacao_setor_bairro ALTER COLUMN NM_MUN SET TAGS ('Censo Demográfico 2022')")
spark.sql("ALTER TABLE IDS_MVP.silver.relacao_setor_bairro ALTER COLUMN NM_BAIRRO SET TAGS ('Censo Demográfico 2022')")
spark.sql("ALTER TABLE IDS_MVP.silver.relacao_setor_bairro ALTER COLUMN NM_FCU SET TAGS ('Censo Demográfico 2022')")

# dom1

spark.sql("COMMENT ON TABLE IDS_MVP.silver.dom1 IS 'Domicilios particulares permanentes ocupados e quantidade de moradores por setor censitario')

spark.sql("ALTER TABLE IDS_MVP.silver.dom1 ALTER COLUMN CD_SETOR COMMENT 'Código do Setor Censitário')
spark.sql("ALTER TABLE IDS_MVP.silver.dom1 ALTER COLUMN VB0001 COMMENT 'Domicilios Particulares Permanentes Ocupados')
spark.sql("ALTER TABLE IDS_MVP.silver.dom1 ALTER COLUMN VB0005 COMMENT 'Domicilios Particulares Permanentes Ocupados, Quantidade de moradores')

spark.sql("ALTER TABLE IDS_MVP.silver.dom1 ALTER COLUMN CD_SETOR SET TAGS ('Censo Demográfico 2022')")
spark.sql("ALTER TABLE IDS_MVP.silver.dom1 ALTER COLUMN VB0001 SET TAGS ('Censo Demográfico 2022')")
spark.sql("ALTER TABLE IDS_MVP.silver.dom1 ALTER COLUMN VB0005 SET TAGS ('Censo Demográfico 2022')")

# dom2

spark.sql("COMMENT ON TABLE IDS_MVP.silver.dom2 IS 'Acesso a agua, esgoto, coleta de lixo e banheiros por setor censitario')

spark.sql("ALTER TABLE IDS_MVP.silver.dom2 ALTER COLUMN CD_SETOR COMMENT 'Código do Setor Censitário')
spark.sql("ALTER TABLE IDS_MVP.silver.dom2 ALTER COLUMN VB0111 COMMENT 'Domicilios Particulares Permanentes Ocupados, Utiliza rede geral de distribuição')
spark.sql("ALTER TABLE IDS_MVP.silver.dom2 ALTER COLUMN VB0309 COMMENT 'Domicilios Particulares Permanentes Ocupados, destinação do esgoto do banheiro ou sanitário ou buraco para dejetos e rede geral ou pluvial')
spark.sql("ALTER TABLE IDS_MVP.silver.dom2 ALTER COLUMN VB0397 COMMENT 'Domicilios Particulares Permanentes Ocupados, Lixo coletado no domicilio por servico de limpeza')
spark.sql("ALTER TABLE IDS_MVP.silver.dom2 ALTER COLUMN VB0398 COMMENT 'Domicilios Particulares Permanentes Ocupados, Lixo depositado em cacamba de servico de limpeza')
spark.sql("ALTER TABLE IDS_MVP.silver.dom2 ALTER COLUMN VB0485 COMMENT 'Domicilios Particulares Permanentes Ocupados, 1 banheiro de uso exclusivo com chuveiro e vaso sanitário existentes no domicilio')
spark.sql("ALTER TABLE IDS_MVP.silver.dom2 ALTER COLUMN VB0486 COMMENT 'Domicilios Particulares Permanentes Ocupados, 2 banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicilio')
spark.sql("ALTER TABLE IDS_MVP.silver.dom2 ALTER COLUMN VB0487 COMMENT 'Domicilios Particulares Permanentes Ocupados, 3 banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicilio')
spark.sql("ALTER TABLE IDS_MVP.silver.dom2 ALTER COLUMN VB0488 COMMENT 'Domicilios Particulares Permanentes Ocupados, 4 banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicilio')
spark.sql("ALTER TABLE IDS_MVP.silver.dom2 ALTER COLUMN VB0489 COMMENT 'Domicilios Particulares Permanentes Ocupados, 5 banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicilio')
spark.sql("ALTER TABLE IDS_MVP.silver.dom2 ALTER COLUMN VB0490 COMMENT 'Domicilios Particulares Permanentes Ocupados, 6 banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicilio')
spark.sql("ALTER TABLE IDS_MVP.silver.dom2 ALTER COLUMN VB0491 COMMENT 'Domicilios Particulares Permanentes Ocupados, 7 banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicilio')
spark.sql("ALTER TABLE IDS_MVP.silver.dom2 ALTER COLUMN VB0492 COMMENT 'Domicilios Particulares Permanentes Ocupados, 8 banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicilio')
spark.sql("ALTER TABLE IDS_MVP.silver.dom2 ALTER COLUMN VB0493 COMMENT 'Domicilios Particulares Permanentes Ocupados, 9 ou mais banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicilio')

Generate (Ctrl + G)
```

Figura 22 - Parte 1 do Código de Preenchimento de Metadados das Tabelas na Camada Silver

```
spark.sql("ALTER TABLE IDS_MVP.silver.dom2 ALTER COLUMN CD_SETOR SET TAGS ('Censo Demográfico 2022')")
spark.sql("ALTER TABLE IDS_MVP.silver.dom2 ALTER COLUMN VB0111 SET TAGS ('Censo Demográfico 2022')")
spark.sql("ALTER TABLE IDS_MVP.silver.dom2 ALTER COLUMN VB0309 SET TAGS ('Censo Demográfico 2022')")
spark.sql("ALTER TABLE IDS_MVP.silver.dom2 ALTER COLUMN VB0397 SET TAGS ('Censo Demográfico 2022')")
spark.sql("ALTER TABLE IDS_MVP.silver.dom2 ALTER COLUMN VB0398 SET TAGS ('Censo Demográfico 2022')")
spark.sql("ALTER TABLE IDS_MVP.silver.dom2 ALTER COLUMN VB0485 SET TAGS ('Censo Demográfico 2022')")
spark.sql("ALTER TABLE IDS_MVP.silver.dom2 ALTER COLUMN VB0486 SET TAGS ('Censo Demográfico 2022')")
spark.sql("ALTER TABLE IDS_MVP.silver.dom2 ALTER COLUMN VB0487 SET TAGS ('Censo Demográfico 2022')")
spark.sql("ALTER TABLE IDS_MVP.silver.dom2 ALTER COLUMN VB0488 SET TAGS ('Censo Demográfico 2022')")
spark.sql("ALTER TABLE IDS_MVP.silver.dom2 ALTER COLUMN VB0489 SET TAGS ('Censo Demográfico 2022')")
spark.sql("ALTER TABLE IDS_MVP.silver.dom2 ALTER COLUMN VB0490 SET TAGS ('Censo Demográfico 2022')")
spark.sql("ALTER TABLE IDS_MVP.silver.dom2 ALTER COLUMN VB0491 SET TAGS ('Censo Demográfico 2022')")
spark.sql("ALTER TABLE IDS_MVP.silver.dom2 ALTER COLUMN VB0492 SET TAGS ('Censo Demográfico 2022')")
spark.sql("ALTER TABLE IDS_MVP.silver.dom2 ALTER COLUMN VB0493 SET TAGS ('Censo Demográfico 2022')")

# alfabetizacao

spark.sql("COMMENT ON TABLE IDS_MVP.silver.alfabetizacao IS 'Alfabetizacao da populacao por faixa etaria em cada setor censitario')

spark.sql("ALTER TABLE IDS_MVP.silver.alfabetizacao ALTER COLUMN CD_SETOR COMMENT 'Código do Setor Censitário')
spark.sql("ALTER TABLE IDS_MVP.silver.alfabetizacao ALTER COLUMN VB0852 COMMENT '15 a 29 anos, Morador sabe ler e escrever')
spark.sql("ALTER TABLE IDS_MVP.silver.alfabetizacao ALTER COLUMN VB0853 COMMENT '15 a 29 anos, Morador não sabe ler e escrever')
spark.sql("ALTER TABLE IDS_MVP.silver.alfabetizacao ALTER COLUMN VB0854 COMMENT '30 a 39 anos, Morador sabe ler e escrever')
spark.sql("ALTER TABLE IDS_MVP.silver.alfabetizacao ALTER COLUMN VB0855 COMMENT '30 a 39 anos, Morador não sabe ler e escrever')
spark.sql("ALTER TABLE IDS_MVP.silver.alfabetizacao ALTER COLUMN VB0856 COMMENT '60 anos ou mais, Morador sabe ler e escrever')
spark.sql("ALTER TABLE IDS_MVP.silver.alfabetizacao ALTER COLUMN VB0857 COMMENT '60 anos ou mais, Morador não sabe ler e escrever')

spark.sql("ALTER TABLE IDS_MVP.silver.alfabetizacao ALTER COLUMN CD_SETOR SET TAGS ('Censo Demográfico 2022')")
spark.sql("ALTER TABLE IDS_MVP.silver.alfabetizacao ALTER COLUMN VB0852 SET TAGS ('Censo Demográfico 2022')")
spark.sql("ALTER TABLE IDS_MVP.silver.alfabetizacao ALTER COLUMN VB0853 SET TAGS ('Censo Demográfico 2022')")
spark.sql("ALTER TABLE IDS_MVP.silver.alfabetizacao ALTER COLUMN VB0854 SET TAGS ('Censo Demográfico 2022')")
spark.sql("ALTER TABLE IDS_MVP.silver.alfabetizacao ALTER COLUMN VB0855 SET TAGS ('Censo Demográfico 2022')")
spark.sql("ALTER TABLE IDS_MVP.silver.alfabetizacao ALTER COLUMN VB0856 SET TAGS ('Censo Demográfico 2022')")
spark.sql("ALTER TABLE IDS_MVP.silver.alfabetizacao ALTER COLUMN VB0857 SET TAGS ('Censo Demográfico 2022')")

# renda

spark.sql("COMMENT ON TABLE IDS_MVP.silver.renda IS 'Rendimento medio mensal das pessoas responsaveis pelos domicilios ocupados')

spark.sql("ALTER TABLE IDS_MVP.silver.renda ALTER COLUMN CD_SETOR COMMENT 'Código do Setor Censitário')
spark.sql("ALTER TABLE IDS_MVP.silver.renda ALTER COLUMN VB0604 COMMENT 'Valor do rendimento nominal médio mensal das pessoas responsáveis com rendimentos por domicilios particulares permanentes ocupados')

spark.sql("ALTER TABLE IDS_MVP.silver.renda ALTER COLUMN CD_SETOR SET TAGS ('Censo Demográfico 2022')")
spark.sql("ALTER TABLE IDS_MVP.silver.renda ALTER COLUMN VB0604 SET TAGS ('Censo Demográfico 2022')")

See performance (S) Checked for improvements
```

Figura 23 - Parte 2 do Código de Preenchimento de Metadados das Tabelas na Camada Silver

Finalmente, falta mais um ponto de qualidade a ser levantado: embora, no Notebook dentro do Databricks realize essa intervenção de qualidade na etapa de trabalho na camada Gold – já que faz parte dos dados que estarão prontos para serem analisados – ela será abordada aqui, pois o autor entende que ela seja mais concernente à etapa de Qualidade de Dados do que necessariamente no capítulo que tratará de Modelagem – naturalmente, o capítulo que trabalhará mais especificamente os processos concernentes à camada Gold. Trata-se da alteração dos valores nulos por zero, em todas as variáveis numéricas de todas as tabelas, pois faz parte da metodologia do IDS realizar essa intervenção, para que alguns setores censitários que possuam informação sobre alguns indicadores – que serão calculados na etapa de modelagem – não tenham seu IDS anulado em razão da ausência de outros indicadores.

```
from pyspark.sql.functions import col, sum as spark_sum, avg, when, coalesce, lit

# salario minimo de 2022
sm_2022 = 1212.0

# Join da silver
relacao = spark.table("IDS_MVP.silver.relacao_setor_bairro")
dom1 = spark.table("IDS_MVP.silver.dom1")
dom2 = spark.table("IDS_MVP.silver.dom2")
alfa = spark.table("IDS_MVP.silver.alfabetizacao")
renda = spark.table("IDS_MVP.silver.renda")

# Join e esquerda: a relacao segura todos os setores
csta_setor = (
    relacao
    .join(dom1, on="CD_SETOR", how="left")
    .join(dom2, on="CD_SETOR", how="left")
    .join(alfa, on="CD_SETOR", how="left")
    .join(renda, on="CD_SETOR", how="left")
)

# Quantidades somas
# V000004: media PRIMEIRO (so nas subdivisoes acima do setor censitario)
agregacoes = [
    spark_sum("V000001").alias("V000001"),
    spark_sum("V000005").alias("V000005"),
    spark_sum("V000011").alias("V000011"),
    spark_sum("V000030").alias("V000030"),
    spark_sum("V000032").alias("V000032"),
    spark_sum("V000038").alias("V000038"),
    spark_sum("V000045").alias("V000045"),
    spark_sum("V000046").alias("V000046"),
    spark_sum("V000047").alias("V000047"),
    spark_sum("V000048").alias("V000048"),
    spark_sum("V000049").alias("V000049"),
    spark_sum("V000040").alias("V000040"),
    spark_sum("V000041").alias("V000041"),
    spark_sum("V000042").alias("V000042"),
    spark_sum("V000043").alias("V000043"),
    spark_sum("V000052").alias("V000052"),
    spark_sum("V000053").alias("V000053"),
    spark_sum("V000054").alias("V000054"),
    spark_sum("V000055").alias("V000055"),
    spark_sum("V000056").alias("V000056"),
    spark_sum("V000057").alias("V000057"),
    avg("V000004").alias("V000004")
]
```

Figura 24 - Parte 1 do Código de substituição de valores nulos por 0.

```
cesta_bairro = (
    cesta_setor
    .groupBy("NM_UF", "NM_MUN", "NM_BAIRRO")
    .agg("agregacoes")
)

cesta_favela = (
    cesta_setor
    .filter(col("NM_FCU").isNotNull())
    .groupBy("NM_UF", "NM_MUN", "NM_FCU")
    .agg("agregacoes")
)

cesta_municipio = (
    cesta_setor
    .groupBy("NM_UF", "NM_MUN")
    .agg("agregacoes")
)

# cestas: as variáveis do Censo + recorte
cestas = {
    "setor": cesta_setor,
    "bairro": cesta_bairro,
    "favela": cesta_favela,
    "municipio": cesta_municipio
}

def calcula_indicadores(df):
    # Substitui valores nulos por zero em todas as colunas numéricas antes das operações matemáticas
    df = df.fillna(0)

    # indicador 1: % água na rede
    df = df.withColumn(
        "indicador_1",
        when(col("V00001") > 0, (col("V00111") / col("V00001")) * 100)
    )

    # indicador 2: % esgoto na rede
    df = df.withColumn(
        "indicador_2",
        when(col("V00001") > 0, (col("V00309") / col("V00001")) * 100)
    )

    # indicador 3: % lixo com coleta adequada
    df = df.withColumn(
        "indicador_3",
        when(col("V00001") > 0, ((col("V00397") + col("V00398")) / col("V00001")) * 100)
    )

    # indicador 4: banheiros por morador
    total_banheiros = (
        coalesce(col("V00405"), lit(0)) * 1 +
        coalesce(col("V00406"), lit(0)) * 2 +
        coalesce(col("V00407"), lit(0)) * 3 +
        coalesce(col("V00408"), lit(0)) * 4 +
        coalesce(col("V00409"), lit(0)) * 5 +
        coalesce(col("V00490"), lit(0)) * 6 +
        coalesce(col("V00491"), lit(0)) * 7 +
        coalesce(col("V00492"), lit(0)) * 8 +
        coalesce(col("V00493"), lit(0)) * 9
    )
    df = df.withColumn(
        "indicador_4",
        when(col("V00001") > 0, total_banheiros / col("V00001"))
    )

    # indicador 5: taxa de analfabetismo (14a)
    nao_le = coalesce(col("V00053"), lit(0)) + coalesce(col("V00055"), lit(0)) + coalesce(col("V00057"), lit(0))
    pop_15mais = (
        coalesce(col("V00052"), lit(0)) +
        coalesce(col("V00053"), lit(0)) +
        coalesce(col("V00054"), lit(0)) +
        coalesce(col("V00055"), lit(0)) +
        coalesce(col("V00056"), lit(0)) +
        coalesce(col("V00057"), lit(0))
    )
    df = df.withColumn(
        "indicador_5",
        when(pop_15mais > 0, (nao_le / pop_15mais) * 100)
    )

    # indicador 6: média da V00004 / salário mínimo
    df = df.withColumn(
        "indicador_6",
        col("V00004") / lit(sm_2022)
    )

    return df
```

Figura 25 - Parte 2 do Código de substituição de valores nulos por 0.

```
# indicador 4: banheiros por morador
total_banheiros = (
    coalesce(col("V00405"), lit(0)) * 1 +
    coalesce(col("V00406"), lit(0)) * 2 +
    coalesce(col("V00407"), lit(0)) * 3 +
    coalesce(col("V00408"), lit(0)) * 4 +
    coalesce(col("V00409"), lit(0)) * 5 +
    coalesce(col("V00490"), lit(0)) * 6 +
    coalesce(col("V00491"), lit(0)) * 7 +
    coalesce(col("V00492"), lit(0)) * 8 +
    coalesce(col("V00493"), lit(0)) * 9
)
df = df.withColumn(
    "indicador_4",
    when(col("V00001") > 0, total_banheiros / col("V00001"))
)

# indicador 5: taxa de analfabetismo (14a)
nao_le = coalesce(col("V00053"), lit(0)) + coalesce(col("V00055"), lit(0)) + coalesce(col("V00057"), lit(0))
pop_15mais = (
    coalesce(col("V00052"), lit(0)) +
    coalesce(col("V00053"), lit(0)) +
    coalesce(col("V00054"), lit(0)) +
    coalesce(col("V00055"), lit(0)) +
    coalesce(col("V00056"), lit(0)) +
    coalesce(col("V00057"), lit(0))
)
df = df.withColumn(
    "indicador_5",
    when(pop_15mais > 0, (nao_le / pop_15mais) * 100)
)

# indicador 6: média da V00004 / salário mínimo
df = df.withColumn(
    "indicador_6",
    col("V00004") / lit(sm_2022)
)

return df
```

Figura 26 - Parte 3 do Código de substituição de valores nulos por 0.

Todos as tabelas possuem como chave principal o código do setor censitário (CD_SETOR), que será utilizado, na etapa de Modelagem, chave de ligação entre as tabelas (basico, alfabetizacao, dom1, dom2 e renda).

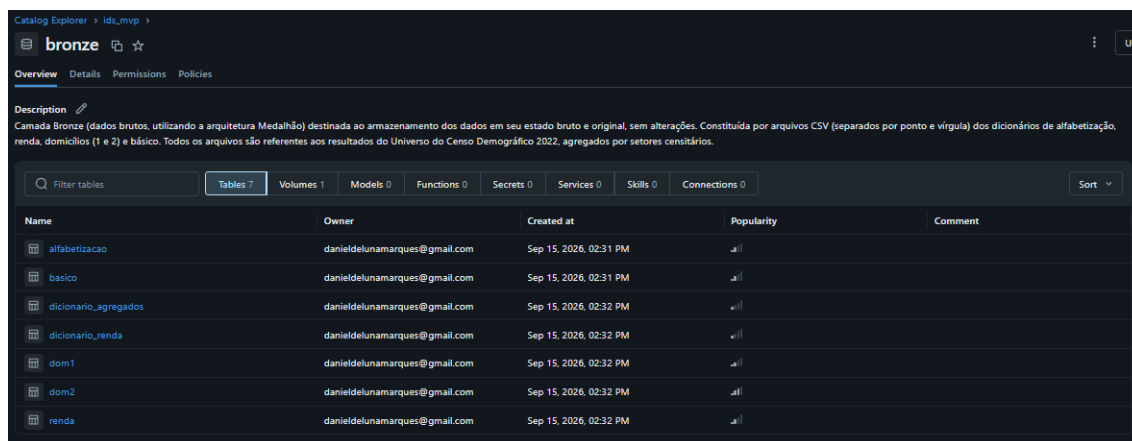
Enfim, encerraram-se as intervenções de análise de qualidade e tratamento dos dados. No próximo capítulo, será apresentada etapa de Modelagem e Catálogo de Dados.

4. Modelagem e Catálogo de Dados

4.1. Catálogo de Dados

Nesta etapa, será exposto o Catálogo de Dados do projeto: como os metadados foram preenchidos através do próprio *Notebook* do *Databricks*, as tabelas e seus atributos serão expostos a partir de *screenshots* do *Unity Catalog* do *Databricks*².

Inicialmente, convém destacar que a “Figura 2 – Overview do Catalog Explorer: IDS_MVP e os schemas criados para as camadas Bronze, Silver e Gold.” apresentada anteriormente, no início deste trabalho, exibe o *Overview* das camadas (ou *schemas*). Como parte significativa das variáveis das tabelas contidas na camada Bronze não foi utilizada – após a seleção feita na Silver –, a imagem abaixo mostrará o conjunto de tabelas delta presentes nesse *schema*, que equivaleria à camada Bronze da arquitetura medalhão:



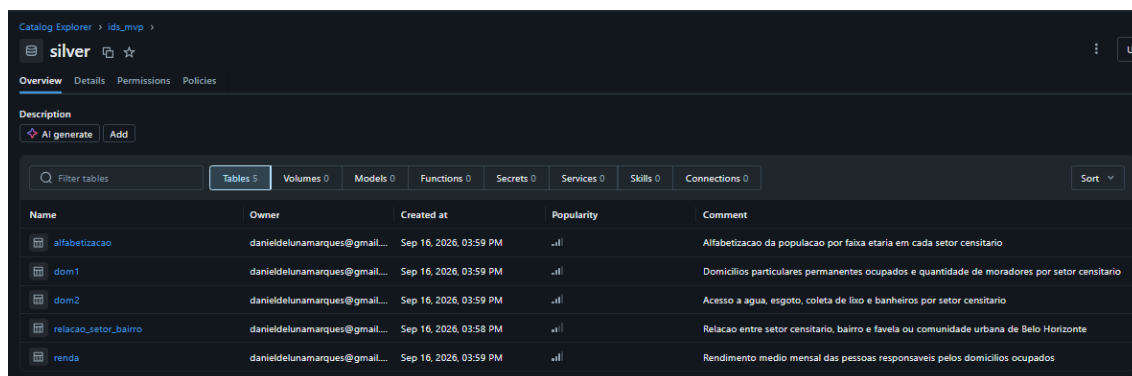
Name	Owner	Created at	Popularity	Comment
alfabetizacao	daniel.lunamarques@gmail.com	Sep 15, 2026, 02:31 PM	▲	
basico	daniel.lunamarques@gmail.com	Sep 15, 2026, 02:31 PM	▲	
dicionario_agregados	daniel.lunamarques@gmail.com	Sep 15, 2026, 02:32 PM	▲	
dicionario_renda	daniel.lunamarques@gmail.com	Sep 15, 2026, 02:32 PM	▲	
dom1	daniel.lunamarques@gmail.com	Sep 15, 2026, 02:32 PM	▲	
dom2	daniel.lunamarques@gmail.com	Sep 15, 2026, 02:32 PM	▲	
renda	daniel.lunamarques@gmail.com	Sep 15, 2026, 02:32 PM	▲	

Figura 27 - Camada Bronze, com as tabelas delta.

Na camada Silver, por sua vez, os dados já passaram por limpeza e tratamento – como exposto anteriormente, na seção Qualidade de Dados. O *overview* desse *schema*,

² De acordo com a consulta feita ao professor Iury Lima Rosal no dia 22/09/2026, foi permitido inserir *screenshots* ao invés de preencher uma tabela com seus atributos.

na forma como foi preenchido a partir das transformações pelas quais as tabelas da camada Bronze passaram, encontra-se na imagem abaixo:



Name	Owner	Created at	Popularity	Comment
alfabetizacao	danieldeunamarques@gmail...	Sep 16, 2026, 03:59 PM	...	Alfabetizacao da populacao por faixa etaria em cada setor censitario
dom1	danieldeunamarques@gmail...	Sep 16, 2026, 03:59 PM	...	Domicilios particulares permanentes ocupados e quantidade de moradores por setor censitario
dom2	danieldeunamarques@gmail...	Sep 16, 2026, 03:59 PM	...	Acesso a agua, esgoto, coleta de lixo e banheiros por setor censitario
relacao_setor_bairro	danieldeunamarques@gmail...	Sep 16, 2026, 03:58 PM	...	Relacao entre setor censitario, bairro e favela ou comunidade urbana de Belo Horizonte
renda	danieldeunamarques@gmail...	Sep 16, 2026, 03:59 PM	...	Rendimento medio mensal das pessoas responsaveis pelos domicilios ocupados

Figura 28 - Overview do schema Silver e suas tabelas, com os metadados preenchidos na forma de comentários

As tabelas, suas variáveis e os respectivos metadados devidamente preenchidos, encontram-se nas imagens posteriores, obedecendo a seguinte ordem: Alfabetização, Domicílios 1, Domicílios 2, Renda e Relação Setor-Bairro.

Uma informação adicional é que as variáveis estão em seu formato devido: com exceção das variáveis de categóricas de recorte geográfico – unidade federativa, município, bairro e nome de favelas ou comunidades urbanas –, da variável que contém o código dos setores censitários, que está como *bigint* e da variável de renda que é do tipo *double*, todas as demais variáveis desta camada são do tipo *int*.

Pós-Graduação em Ciência de Dados e Analytics
Sprint de Engenharia de Dados
Aluno: Daniel de Luna Marques
Matrícula: 4052026000228



Catalog Explorer > ids_mvp > silver >

alfabetizacao ☆

Overview Sample Data Details Permissions Policies History Lineage Insights Quality

Description Alfabetizacao da populacao por faixa etaria em cada setor censitario

Filter columns...

Column	Type	Popularity	Comment	Tags	Column masking	
CD_SETOR	bigint		Código do Setor Censitário	Censo Demográfico 2022		
V00852	int		15 a 29 anos, Morador sabe ler e escrever	Censo Demográfico 2022		
V00853	int		15 a 29 anos, Morador não sabe ler e escrever	Censo Demográfico 2022		
V00854	int		30 a 59 anos, Morador sabe ler e escrever	Censo Demográfico 2022		
V00855	int		30 a 59 anos, Morador não sabe ler e escrever	Censo Demográfico 2022		
V00856	int		60 anos ou mais, Morador sabe ler e escrever	Censo Demográfico 2022		
V00857	int		60 anos ou mais, Morador não sabe ler e escrever	Censo Demográfico 2022		

Figura 29- *Overview* da tabela de Alfabetização na camada Silver e seus metadados.

Catalog Explorer > ids_mvp > silver >

dom1 ☆

Overview Sample Data Details Permissions Policies History Lineage Insights Quality

Description Domicilios particulares permanentes ocupados e quantidade de moradores por setor censitario

Filter columns...

Column	Type	Popularity	Comment	Tags	Column masking	
CD_SETOR	bigint		Código do Setor Censitário	Censo Demográfico 2022		
V00001	int		Domicilios Particulares Permanentes Ocupados	Censo Demográfico 2022		
V00005	int		Domicilios Particulares Permanentes Ocupados, Quantidade de moradores	Censo Demográfico 2022		

Figura 30 - *Overview* da tabela Domicílios 1 na camada Silver e seus metadados.

Pós-Graduação em Ciência de Dados e Analytics
Sprint de Engenharia de Dados
Aluno: Daniel de Luna Marques
Matrícula: 4052026000228

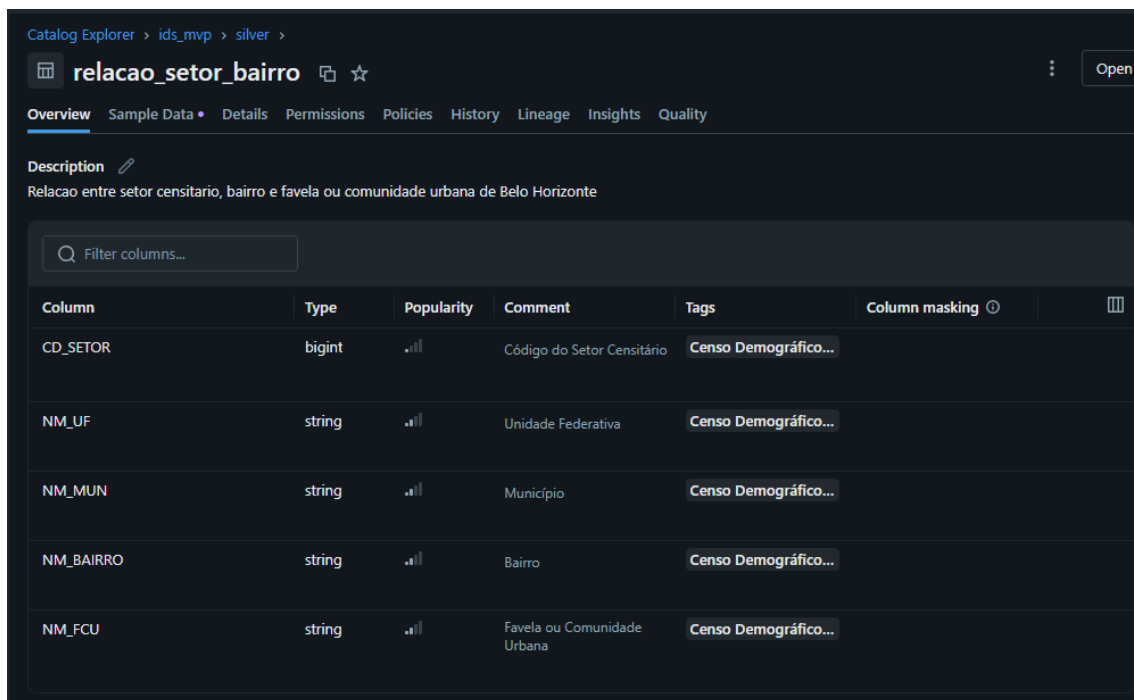


Column	Type	Popularity	Comment	Tags	Column masking
CD_SETOR	bigint	...	Código do Setor Censitário	Censo Demográfico 2022	
V00111	int	...	Domicílios Particulares Permanentes Ocupados, Utiliza rede geral de distribuição	Censo Demográfico 2022	
V00399	int	...	Domicílios Particulares Permanentes Ocupados, Destinação do esgoto do banheiro ou sanitário no banheiro para esgotos e rede geral ou pluvial	Censo Demográfico 2022	
V00397	int	...	Domicílios Particulares Permanentes Ocupados, Lixo coletado no domicílio por serviço de limpeza	Censo Demográfico 2022	
V00398	int	...	Domicílios Particulares Permanentes Ocupados, Lixo depositado em caçamba de serviço de limpeza	Censo Demográfico 2022	
V00485	int	...	Domicílios Particulares Permanentes Ocupados, 1 banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicílio	Censo Demográfico 2022	
V00488	int	...	Domicílios Particulares Permanentes Ocupados, 2 banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicílio	Censo Demográfico 2022	
V00487	int	...	Domicílios Particulares Permanentes Ocupados, 3 banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicílio	Censo Demográfico 2022	
V00488	int	...	Domicílios Particulares Permanentes Ocupados, 4 banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicílio	Censo Demográfico 2022	
V00489	int	...	Domicílios Particulares Permanentes Ocupados, 5 banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicílio	Censo Demográfico 2022	
V00490	int	...	Domicílios Particulares Permanentes Ocupados, 6 banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicílio	Censo Demográfico 2022	
V00491	int	...	Domicílios Particulares Permanentes Ocupados, 7 banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicílio	Censo Demográfico 2022	
V00492	int	...	Domicílios Particulares Permanentes Ocupados, 8 banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicílio	Censo Demográfico 2022	
V00493	int	...	Domicílios Particulares Permanentes Ocupados, 9 ou mais banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicílio	Censo Demográfico 2022	

Figura 31 - *Overview* da tabela Domicílios 2 na camada Silver e seus metadados.

Column	Type	Popularity	Comment	Tags	Column masking
CD_SETOR	bigint	...	Código do Setor Censitário	Censo Demográfico 2022	
V06004	float	...	Valor do rendimento nominal médio mensal das pessoas responsáveis com rendimentos por domicílios particulares permanentes ocupados	Censo Demográfico 2022	

Figura 32 - *Overview* da tabela de Renda na camada Silver e seus metadados.



Catalog Explorer > ids_mvp > silver >

relacao_setor_bairro ☆

Overview Sample Data Details Permissions Policies History Lineage Insights Quality

Description ✎
Relacao entre setor censitario, bairro e favela ou comunidade urbana de Belo Horizonte

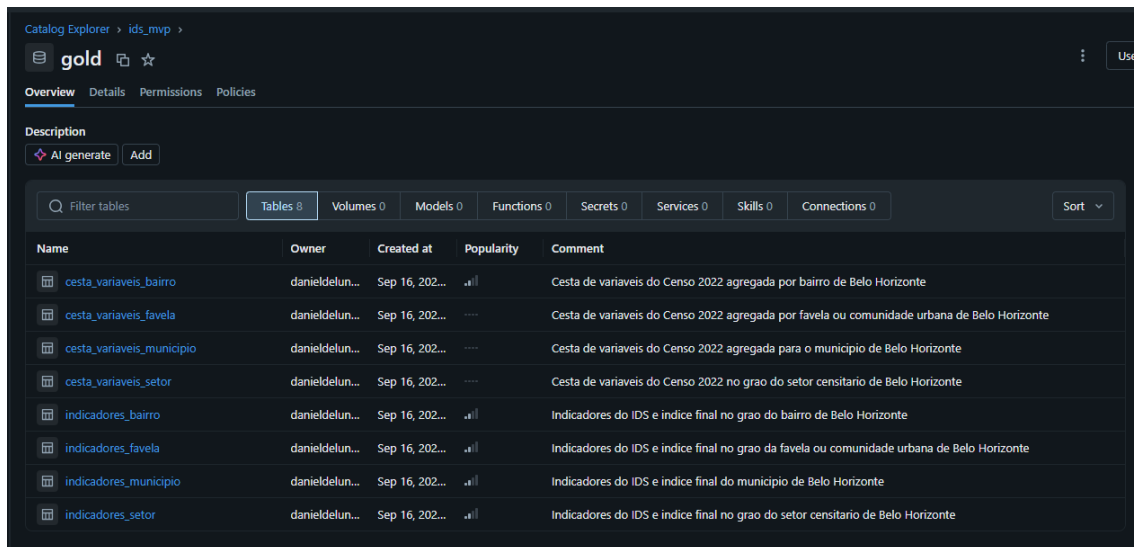
Filter columns...

Column	Type	Popularity	Comment	Tags	Column masking ⓘ	⌵
CD_SETOR	bigint	...	Código do Setor Censitário	Censo Demográfico...		
NM_UF	string	...	Unidade Federativa	Censo Demográfico...		
NM_MUN	string	...	Município	Censo Demográfico...		
NM_BAIRRO	string	...	Bairro	Censo Demográfico...		
NM_FCU	string	...	Favela ou Comunidade Urbana	Censo Demográfico...		

Figura 33 - *Overview* da tabela de Relação Setor-Bairro na camada Silver e seus metadados.

Todas essas tabelas e suas variáveis foram utilizadas como insumos para o cálculo do IDS, diferentemente das tabelas da camada Bronze, cuja maior parte das variáveis, foi descartada. Esses cálculos serão demonstrados no subtítulo de Modelagem.

Antes, contudo, de chegar à Modelagem, importa apresentar o catálogo dos dados gerados nessa etapa, cujos processos concernem integralmente à camada Gold.



Name	Owner	Created at	Popularity	Comment
cesta_variaveis_bairro	danieldelun...	Sep 16, 202...	...	Cesta de variaveis do Censo 2022 agregada por bairro de Belo Horizonte
cesta_variaveis_favela	danieldelun...	Sep 16, 202...	...	Cesta de variaveis do Censo 2022 agregada por favela ou comunidade urbana de Belo Horizonte
cesta_variaveis_municipio	danieldelun...	Sep 16, 202...	...	Cesta de variaveis do Censo 2022 agregada para o municipio de Belo Horizonte
cesta_variaveis_setor	danieldelun...	Sep 16, 202...	...	Cesta de variaveis do Censo 2022 no grao do setor censitario de Belo Horizonte
indicadores_bairro	danieldelun...	Sep 16, 202...	...	Indicadores do IDS e indice final no grao do bairro de Belo Horizonte
indicadores_favela	danieldelun...	Sep 16, 202...	...	Indicadores do IDS e indice final no grao da favela ou comunidade urbana de Belo Horizonte
indicadores_municipio	danieldelun...	Sep 16, 202...	...	Indicadores do IDS e indice final do municipio de Belo Horizonte
indicadores_setor	danieldelun...	Sep 16, 202...	...	Indicadores do IDS e indice final no grao do setor censitario de Belo Horizonte

Figura 34 - Overview do schema Gold e suas tabelas, com os metadados preenchidos na forma de comentários.

Nesse *schema*, as tabelas são do tipo “Cestas de Variáveis” ou “Indicadores”, todas elas agregadas para todos os níveis intramunicipais analisados nesse projeto. As Cestas consistem em tabelas que possuem todas as variáveis presentes nas tabelas temáticas da camada Silver, agregadas pelo nível territorial trabalho. Já as tabelas com os indicadores, possuem os indicadores calculados a partir dessas variáveis, da maneira como preconiza a metodologia do IDS – que será exposta na etapa de Modelagem.

Portanto, todas as tabelas de Cestas possuem a mesma estrutura, com a única diferença de que são agregadas por níveis territoriais distintos: setores censitários, bairros, favelas e o próprio município. E isso é válido para as tabelas de Indicadores. Desta forma, com o fim de evitar redundâncias que não agreguem valor para este trabalho, o autor apresentará os *screenshots* dos níveis de agregação por setores censitários, tomando-os como suficientes para comprovar a estrutura, afinal, as demais tabelas foram agregadas a partir deste nível territorial.

Pós-Graduação em Ciência de Dados e Analytics
Sprint de Engenharia de Dados
Aluno: Daniel de Luna Marques
Matrícula: 4052026000228



Catalog Explorer > ids_mvp > gold >

cesta_variaveis_setor

Overview Sample Data Details Permissions Policies History Lineage Insights Quality

Description

Cesta de variáveis do Censo 2022 no grau do setor censitário de Belo Horizonte

Filter columns...

Column	Type	Popularity	Comment	Tags	Column masking
CD_SETOR	bigint	---	Código do Setor Censitário	Censo Demográfico 2022	
NM_UF	string	---	Unidade Federativa	Censo Demográfico 2022	
NM_MUN	string	---	Município	Censo Demográfico 2022	
NM_BAIRRO	string	---	Bairro	Censo Demográfico 2022	
NM_FCU	string	---	Favela ou Comunidade Urbana	Censo Demográfico 2022	
V00001	int	---	Domicílios Particulares Permanentes Ocupados	Censo Demográfico 2022	
V00005	int	---	Domicílios Particulares Permanentes Ocupados, Quantidade de moradores	Censo Demográfico 2022	
V00111	int	---	Domicílios Particulares Permanentes Ocupados, Utiliza rede geral de distribuição	Censo Demográfico 2022	
V00309	int	---	Domicílios Particulares Permanentes Ocupados, Destinação do esgoto do banheiro do sanitário ou banheiro para despejo: é rede geral ou pluvial	Censo Demográfico 2022	
V00397	int	---	Domicílios Particulares Permanentes Ocupados, Lixo coletado no domicílio por serviço de limpeza	Censo Demográfico 2022	
V00398	int	---	Domicílios Particulares Permanentes Ocupados, Lixo depositado em caçamba de serviço de limpeza	Censo Demográfico 2022	
V00485	int	---	Domicílios Particulares Permanentes Ocupados, 1 banheiro de uso exclusivo com chuveiro e vaso sanitário existentes no domicílio	Censo Demográfico 2022	

Figura 35 - Parte 1 do *Overview* da tabela de Cestas de Variáveis agregados por setores censitários, na camada Gold.

V00486	int	---	Domicílios Particulares Permanentes Ocupados, 2 banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicílio	Censo Demográfico 2022
V00487	int	---	Domicílios Particulares Permanentes Ocupados, 3 banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicílio	Censo Demográfico 2022
V00488	int	---	Domicílios Particulares Permanentes Ocupados, 4 banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicílio	Censo Demográfico 2022
V00489	int	---	Domicílios Particulares Permanentes Ocupados, 5 banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicílio	Censo Demográfico 2022
V00490	int	---	Domicílios Particulares Permanentes Ocupados, 6 banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicílio	Censo Demográfico 2022
V00491	int	---	Domicílios Particulares Permanentes Ocupados, 7 banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicílio	Censo Demográfico 2022
V00492	int	---	Domicílios Particulares Permanentes Ocupados, 8 banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicílio	Censo Demográfico 2022
V00493	int	---	Domicílios Particulares Permanentes Ocupados, 9 ou mais banheiros de uso exclusivo com chuveiro e vaso sanitário existentes no domicílio	Censo Demográfico 2022

Figura 36 – Parte 2 do *Overview* da tabela de Cestas de Variáveis agregados por setores censitários, na camada Gold.

Pós-Graduação em Ciência de Dados e Analytics
Sprint de Engenharia de Dados
Aluno: Daniel de Luna Marques
Matrícula: 4052026000228



Catalog Explorer > ids_mvp > gold > cesta_variaveis_setor

Overview Sample Data Details Permissions Policies History Lineage Insights Quality

Description
Cesta de variáveis do Censo 2022 no grao do setor censitário de Belo Horizonte

Filter columns...

Column	Type	Popularity	Comment	Tags	Column masking
V00852	int	---	15 a 29 anos, Morador sabe ler e escrever	Censo Demográfico 2022	
V00853	int	---	15 a 29 anos, Morador não sabe ler e escrever	Censo Demográfico 2022	
V00854	int	---	30 a 59 anos, Morador sabe ler e escrever	Censo Demográfico 2022	
V00855	int	---	30 a 59 anos, Morador não sabe ler e escrever	Censo Demográfico 2022	
V00856	int	---	60 anos ou mais, Morador sabe ler e escrever	Censo Demográfico 2022	
V00857	int	---	60 anos ou mais, Morador não sabe ler e escrever	Censo Demográfico 2022	
V06004	float	---	Valor do rendimento nominal médio mensal das pessoas responsáveis com rendimentos por domicílios particulares permanentes ocupados	Censo Demográfico 2022	

< 1 2

Figura 37 - Parte 3 do *Overview* da tabela de Cestas de Variáveis agregados por setores censitários, na camada Gold.

Finalmente, essas variáveis foram utilizadas para calcular os indicadores que, mais tarde foram normalizados e, após isso, usados para calcular o IDS para os diferentes recortes intramunicipais.

Catalog Explorer > ids_mvp > gold > indicadores_setor

Overview Sample Data Details Permissions Policies History Lineage Insights Quality

Description
Indicadores do IDS e índice final no grao do setor censitário de Belo Horizonte

Filter columns...

Column	Type	Popularity	Comment	Tags	Column masking
CD_SETOR	bigint	✓	Código do Setor Censitário	Censo Demográfico 2022	
NM_UF	string	---	Unidade Federativa	Censo Demográfico 2022	
NM_MUN	string	---	Município	Censo Demográfico 2022	
NM_BAIRRO	string	✓	Bairro	Censo Demográfico 2022	
NM_FCU	string	✓	Favela ou Comunidade Urbana	Censo Demográfico 2022	
Indicador_1	double	✓	Indicador 1 - Forma adequada de abastecimento de água (%)	IDS	
Indicador_2	double	✓	Indicador 2 - Esgotamento sanitário adequado (%)	IDS	
Indicador_3	double	✓	Indicador 3 - Coleta de lixo adequada (%)	IDS	
Indicador_4	double	✓	Indicador 4 - Habitabilidade (banheiros por pessoa)	IDS	
Indicador_5	double	✓	Indicador 5 - Taxa de analfabetismo (15 anos ou mais) (%)	IDS	
Indicador_6	double	✓	Indicador 6 - Rendimento médio do responsável em salários mínimos de 2022	IDS	
Indicador_1_norm	double	✓	Indicador 1 normalizado - Forma adequada de abastecimento de água (%)	IDS	
Indicador_2_norm	double	✓	Indicador 2 normalizado - Esgotamento sanitário adequado (%)	IDS	
Indicador_3_norm	double	✓	Indicador 3 normalizado - Coleta de lixo adequada (%)	IDS	
Indicador_4_norm	double	✓	Indicador 4 normalizado - Habitabilidade (banheiros por pessoa)	IDS	
Indicador_5_norm	double	✓	Indicador 5 normalizado - Rendimento médio do responsável em salários mínimos de 2022	IDS	
Indicador_6_norm	double	✓	Indicador 6 normalizado (invertido) - Taxa de analfabetismo (15 anos ou mais) (%)	IDS	
ids	double	✓	Índice de Desenvolvimento Social (média dos 6 indicadores normalizados)	IDS	

Figura 38 - *Overview* da tabela de Cestas de Variáveis agregados por setores censitários, na camada Gold.

Com base na imagem acima, todos os indicadores constam como *double*, pois todos eles possuem valores decimais: os 5 primeiros indicadores são porcentagens, enquanto o 6º -- renda média dos responsáveis em domicílios particulares permanentes ocupados -- está como valor decimal. Os indicadores são normalizados para ficarem em uma escala situada entre 0 e 1 – como será visto na seção seguinte –, motivo pelo qual também estão em *double*.

Finalmente, o IDS, que é a média entre esses indicadores normalizados também está *double*. Assim como os outros dados apresentados nessa seção, essa tabela de indicadores – bem como as outras, agregadas por outros níveis territoriais – também conta com todos os metadados devidamente preenchidos: cada variável possui a explicação na forma de comentário. Neste trabalho, a apresentação do significado desses valores constará a na próxima seção.

4.2. Modelagem

Nesta etapa, a modelagem foi estruturada para refletir as necessidades de análise geográfica exigidas pelas perguntas de negócio. Inicialmente, realizou-se a consolidação das informações a partir de um cruzamento de dados (*join à esquerda*) tomando como base a tabela de referência *relacao_setor_bairro*, garantindo a manutenção da malha censitária territorial para o vínculo com as tabelas temáticas *alfabetizacao*, *dom1*, *dom2* e *renda* já tratadas na camada Silver.

Com as bases unificadas, procedeu-se à criação das "Cestas de Variáveis", que representam os dados agregados necessários para a construção dos indicadores finais. A modelagem determinou a criação de quatro níveis territoriais distintos na camada Gold: por setores censitários (granulação mínima), por bairros, por favelas ou comunidades urbanas e, por fim, para a totalidade do município de Belo Horizonte. Na mecânica de agrupamento (*groupBy*), a regra de negócio estabeleceu que as quantificações populacionais e domiciliares fossem agregadas por meio de soma, enquanto a variável

correspondente à renda dos responsáveis (V06004) exigiu a aplicação da média aritmética.

```
from pyspark.sql.functions import col, sum as spark_sum, avg

# leio da silver o que ja esta limpo e tipado
relacao = spark.table("IDS_MVP.silver.relacao_setor_bairro")
dom1 = spark.table("IDS_MVP.silver.dom1")
dom2 = spark.table("IDS_MVP.silver.dom2")
alfa = spark.table("IDS_MVP.silver.alfabetizacao")
renda = spark.table("IDS_MVP.silver.renda")

# 3.1.1 join a esquerda
# a relacao fica a esquerda para nao perder setor
# setor sem morador entra com as V... nulas

cesta_setor = (
    relacao
    .join(dom1, on="CD_SETOR", how="left")
    .join(dom2, on="CD_SETOR", how="left")
    .join(alfa, on="CD_SETOR", how="left")
    .join(renda, on="CD_SETOR", how="left")
)

# 3.1.2 o que entra no groupby
# quantidade (domicilio, morador, banheiro, alfabetizacao): soma
# V06004 (renda media ja calculada no setor): media

agregacoes = [
    spark_sum("V06001").alias("V06001"),
    spark_sum("V06005").alias("V06005"),
    spark_sum("V06111").alias("V06111"),
    spark_sum("V06302").alias("V06302"),
    spark_sum("V06397").alias("V06397"),
    spark_sum("V06398").alias("V06398"),
    spark_sum("V06485").alias("V06485"),
    spark_sum("V06486").alias("V06486"),
    spark_sum("V06487").alias("V06487"),
    spark_sum("V06488").alias("V06488"),
    spark_sum("V06489").alias("V06489"),
    spark_sum("V06490").alias("V06490"),
    spark_sum("V06491").alias("V06491"),
    spark_sum("V06492").alias("V06492"),
    spark_sum("V06493").alias("V06493"),
    spark_sum("V06852").alias("V06852"),
    spark_sum("V06853").alias("V06853"),
    spark_sum("V06854").alias("V06854"),
    spark_sum("V06855").alias("V06855"),
    spark_sum("V06856").alias("V06856"),
    spark_sum("V06857").alias("V06857"),
    avg("V06004").alias("V06004")
]
```

Figura 39 - Parte 1 código da junção das tabelas da camada Silver.

```
}

# bairro
cesta_bairro = (
    cesta_setor
    .groupBy("NM_UF", "NM_MUN", "NM_BAIRRO")
    .agg(*agregacoes)
)

# favela
# setor sem NM_FCU nao e favela, por isso o filtro
cesta_favela = (
    cesta_setor
    .filter(col("NM_FCU").isNull())
    .groupBy("NM_UF", "NM_MUN", "NM_FCU")
    .agg(*agregacoes)
)

# municipio
cesta_municipio = (
    cesta_setor
    .groupBy("NM_UF", "NM_MUN")
    .agg(*agregacoes)
)

# as quatro cestas no mesmo dicionario
# chave = subdivisao territorial
cestas = {
    "setor": cesta_setor,
    "bairro": cesta_bairro,
    "favela": cesta_favela,
    "municipio": cesta_municipio
}

> 9 dataframes: alfa, cesta_bairro, cesta_favela, cesta_municipio, cesta_setor, dom1, ...
```

Figura 40 - Parte 2 código da junção das tabelas da camada Silver.

Após a consolidação das Cestas de Variáveis nos diferentes níveis hierárquicos territoriais, iniciou-se a fase de cálculo dos seis indicadores que compõem o Índice de Desenvolvimento Social (IDS). Neste ponto da modelagem, executou-se uma regra de tratamento de negócio importante: a substituição matemática de todos os valores nulos – convertidos anteriormente na camada de Qualidade de Dados devido às regras de sigilo do IBGE – por zeros. Essa manobra metodológica foi estritamente necessária para viabilizar as fórmulas (como divisões e percentuais) sem provocar erros de execução e garantindo que nenhum recorte intramunicipal tivesse o seu IDS anulado por escassez pontual de dados em uma variável isolada.

Neste momento da modelagem, faz-se necessário referenciar a metodologia do IDS, na forma como ela propõe a construção dos indicadores:

1. **Forma adequada de abastecimento de água** Percentagem de domicílios particulares permanentes ocupados ligados à rede geral de distribuição sobre o total de domicílios particulares permanentes ocupados.

$$Indicador\ 1 = \left(\frac{V00111}{V00001} \right) \times 100$$

2. **Esgotamento sanitário adequado** Percentagem de domicílios particulares permanentes ocupados com esgoto ligado à rede geral de esgoto ou à rede pluvial sobre o total de domicílios particulares permanentes ocupados.

$$Indicador\ 2 = \left(\frac{V00309}{V00001} \right) \times 100$$

3. **Coleta de lixo adequada** Percentagem de domicílios particulares permanentes ocupados com lixo coletado diretamente por serviço de limpeza ou depositado em caçamba de serviço de limpeza sobre o total de domicílios particulares permanentes ocupados.

$$\text{Indicador 3} = \left(\frac{V00397 + V00398}{V00001} \right) \times 100$$

4. **Habitabilidade (Banheiros por pessoa)** Número médio de banheiros por pessoa, correspondente à razão entre o total de banheiros de uso exclusivo (com chuveiro e vaso sanitário) e o número total de moradores do domicílio.

$$\text{Indicador 4} = \frac{V00485 + (V00486 \times 2) + \dots + (V00493 \times 9)}{V00005}$$

Nota: O numerador soma a quantidade total de banheiros, multiplicando o número de domicílios pela quantidade de banheiros que possuem (de 1 até 9 ou mais).

5. **Taxa de analfabetismo (15 anos ou mais)** Percentagem de moradores de 15 anos e mais de idade que não sabem ler e escrever em relação ao total de moradores dessa mesma faixa etária.

$$\text{Indicador 5} = \left(\frac{V00853 + V00855 + V00857}{V00852 + V00853 + V00854 + V00855 + V00856 + V00857} \right) \times 100$$

6. **Rendimento médio do responsável** Rendimento médio nominal mensal do responsável pelo domicílio particular permanente ocupado, excluindo-se da média os responsáveis declarados "sem rendimento", expresso em salários mínimos (o salário mínimo para 2022 foi de R\$1212,00. [Veja o texto integral da Lei nº 14.358, de 1º de junho de 2022](#)).

$$\text{Indicador}_6 = V06004/1212$$

Esses cálculos foram realizados no script na forma como está estruturado abaixo:

```
from pyspark.sql.functions import col, sum as spark_sum, avg, when, coalesce, lit

# salario minimo de 2022
sm_2022 = 1212.0

# Join da silver
relacao = spark.table("IDS_MVP.silver.relacao_setor_bairro")
dom1 = spark.table("IDS_MVP.silver.dom1")
dom2 = spark.table("IDS_MVP.silver.dom2")
alfa = spark.table("IDS_MVP.silver.alfabetizacao")
renda = spark.table("IDS_MVP.silver.renda")

# Join a esquerda: a relacao segura todos os setores
cesta_setor = (
    relacao
    .join(dom1, on="CD_SETOR", how="left")
    .join(dom2, on="CD_SETOR", how="left")
    .join(alfa, on="CD_SETOR", how="left")
    .join(renda, on="CD_SETOR", how="left")
)

# quantidades somas
# V06004: media PRIMEIRO (so nas subdivisoes acima do setor censitario)
agregacoes = [
    spark_sum("V06001").alias("V06001"),
    spark_sum("V06005").alias("V06005"),
    spark_sum("V06111").alias("V06111"),
    spark_sum("V06389").alias("V06389"),
    spark_sum("V06397").alias("V06397"),
    spark_sum("V06398").alias("V06398"),
    spark_sum("V06455").alias("V06455"),
    spark_sum("V06482").alias("V06482"),
    spark_sum("V06487").alias("V06487"),
    spark_sum("V06488").alias("V06488"),
    spark_sum("V06489").alias("V06489"),
    spark_sum("V06490").alias("V06490"),
    spark_sum("V06492").alias("V06492"),
    spark_sum("V06493").alias("V06493"),
    spark_sum("V06494").alias("V06494"),
    spark_sum("V06511").alias("V06511"),
    spark_sum("V06544").alias("V06544"),
    spark_sum("V06551").alias("V06551"),
    spark_sum("V06554").alias("V06554"),
    spark_sum("V06557").alias("V06557"),
    avg("V06004").alias("V06004")
]

cesta_bairro = (
    cesta_setor
    .groupBy("NM_UF", "NM_MUN", "NM_BAIRRO")
    .agg(*agregacoes)
)

cesta_favela = (
    cesta_setor
```

Figura 41 - Parte 1 Código de construção de Cesta de Variáveis do IDS

```
cesta_setor =  
    .filter(col("NM_FCU").isNotNull())  
    .groupBy("NM_UF", "NM_MUN", "NM_FCU")  
    .agg("agregacoes")  
)  
  
cesta_municipio = (  
    cesta_setor  
    .groupBy("NM_UF", "NM_MUN")  
    .agg("agregacoes")  
)  
  
# Cestas: só variáveis do Censo + recorta  
cestas = {  
    "setor": cesta_setor,  
    "bairro": cesta_bairro,  
    "favela": cesta_favela,  
    "municipio": cesta_municipio  
}  
  
def calcula_indicadores(df):  
    # substitui valores nulos por zero em todas as colunas numéricas antes das operações estatísticas  
    df = df.fillna(0)  
  
    # indicador 1: % água na rede  
    df = df.withColumn(  
        "indicador_1",  
        when(col("V00001") > 0, (col("V00111") / col("V00001")) * 100)  
    )  
  
    # indicador 2: % esgoto na rede  
    df = df.withColumn(  
        "indicador_2",  
        when(col("V00001") > 0, (col("V00309") / col("V00001")) * 100)  
    )  
  
    # indicador 3: % lixo com coleta adequada  
    df = df.withColumn(  
        "indicador_3",  
        when(col("V00001") > 0, ((col("V00397") + col("V00398")) / col("V00001")) * 100)  
    )  
  
    # indicador 4: banheiros por morador  
    total_banheiros = (  
        coalesce(col("V00485"), lit(0)) + 1 +  
        coalesce(col("V00486"), lit(0)) + 2 +  
        coalesce(col("V00487"), lit(0)) + 3 +  
        coalesce(col("V00488"), lit(0)) + 4 +  
        coalesce(col("V00489"), lit(0)) + 5 +  
        coalesce(col("V00490"), lit(0)) + 6 +  
        coalesce(col("V00491"), lit(0)) + 7 +  
        coalesce(col("V00492"), lit(0)) + 8 +  
        coalesce(col("V00493"), lit(0)) + 9  
    )  
  
    df = df.withColumn(  
        "indicador_4",  
        when(col("V00001") > 0, total_banheiros / col("V00001"))  
    )
```

Figura 42 - Parte 2 Código de construção de Cesta de Variáveis do IDS

```
)  
  
# indicador 5: taxa de analfabetismo (15+)  
nao_le = coalesce(col("V00051"), lit(0)) + coalesce(col("V00055"), lit(0)) + coalesce(col("V00057"), lit(0))  
pop_15mais = (  
    coalesce(col("V00052"), lit(0)) +  
    coalesce(col("V00053"), lit(0)) +  
    coalesce(col("V00054"), lit(0)) +  
    coalesce(col("V00055"), lit(0)) +  
    coalesce(col("V00056"), lit(0)) +  
    coalesce(col("V00057"), lit(0))  
)  
  
df = df.withColumn(  
    "indicador_5",  
    when(pop_15mais > 0, (nao_le / pop_15mais) * 100)  
)  
  
# indicador 6: renda da V00004 / salário mínimo  
df = df.withColumn(  
    "indicador_6",  
    col("V00004") / lit(5022)  
)  
  
return df  
  
# calcula em cima da cesta  
# indicadores: subdivisões intramunicipais + os 6 indicadores  
indicadores = {  
    "setor": calcula_indicadores(cestas["setor"]).select(  
        "CD_SETOR", "NM_UF", "NM_MUN", "NM_BAIRRO", "NM_FCU",  
        "indicador_1", "indicador_2", "indicador_3",  
        "indicador_4", "indicador_5", "indicador_6"  
    ),  
    "bairro": calcula_indicadores(cestas["bairro"]).select(  
        "NM_UF", "NM_MUN", "NM_BAIRRO",  
        "indicador_1", "indicador_2", "indicador_3",  
        "indicador_4", "indicador_5", "indicador_6"  
    ),  
    "favela": calcula_indicadores(cestas["favela"]).select(  
        "NM_UF", "NM_MUN", "NM_FCU",  
        "indicador_1", "indicador_2", "indicador_3",  
        "indicador_4", "indicador_5", "indicador_6"  
    ),  
    "municipio": calcula_indicadores(cestas["municipio"]).select(  
        "NM_UF", "NM_MUN",  
        "indicador_1", "indicador_2", "indicador_3",  
        "indicador_4", "indicador_5", "indicador_6"  
    )  
}  
  
> @dataframe: alpha_cesta_bairro_cesta_favela_cesta_municipio_cesta_setor_dom1...
```

Figura 43- Parte 3 Código de construção de Cesta de Variáveis do IDS

Para que esses seis indicadores recém-calculados pudessem ser consolidados em um índice sintético (o IDS final), aplicou-se a técnica de normalização matemática, parametrizando todos eles para uma escala unificada de valor contínuo variando de 0 a 1. Para isso, foi necessário realizar a extração unificada dos limites (valores máximos e mínimos globais) de cada indicador, processando simultaneamente todos os níveis territoriais.

```
38
from pyspark.sql.functions import col, lit, min as spark_min, max as spark_max

# 1) espelha os 4 recortes no padrão da tabela de referência
# nível, id_recorte, nm_recorte + indicadores

ind_setor = {
    indicadores["setor"]
    .select(
        lit("setor").alias("nível"),
        col("CD_SETOR").cast("string").alias("id_recorte"),
        col("CD_SETOR").cast("string").alias("nm_recorte"),
        "indicador_1", "indicador_2", "indicador_3",
        "indicador_4", "indicador_5", "indicador_6"
    )
}

ind_bairro = {
    indicadores["bairro"]
    .select(
        lit("bairro").alias("nível"),
        col("NM_BAIRRO").cast("string").alias("id_recorte"),
        col("NM_BAIRRO").cast("string").alias("nm_recorte"),
        "indicador_1", "indicador_2", "indicador_3",
        "indicador_4", "indicador_5", "indicador_6"
    )
}

ind_favela = {
    indicadores["favela"]
    .select(
        lit("favela").alias("nível"),
        col("NM_FCU").cast("string").alias("id_recorte"),
        col("NM_FCU").cast("string").alias("nm_recorte"),
        "indicador_1", "indicador_2", "indicador_3",
        "indicador_4", "indicador_5", "indicador_6"
    )
}

ind_municipio = {
    indicadores["municipio"]
    .select(
        lit("municipio").alias("nível"),
        col("NM_MUN").cast("string").alias("id_recorte"),
        col("NM_MUN").cast("string").alias("nm_recorte"),
        "indicador_1", "indicador_2", "indicador_3",
        "indicador_4", "indicador_5", "indicador_6"
    )
}

indicadores_unificado = (
    ind_setor
    .unionByName(ind_bairro)
    .unionByName(ind_favela)
)
```

Figura 44 - Parte 1 do Código de construção da tabela de valores máximos e mínimos para a normalização.

```

.unionByName(ind_favela)
.unionByName(ind_municipio)

# 2) min e max de cada indicador, por nível

ref_minmax = (
    indicadores_unificado
    .groupBy("nível")
    .agg(
        spark_min("indicador_1").alias("min_indicador_1"),
        spark_max("indicador_1").alias("max_indicador_1"),
        spark_min("indicador_2").alias("min_indicador_2"),
        spark_max("indicador_2").alias("max_indicador_2"),
        spark_min("indicador_3").alias("min_indicador_3"),
        spark_max("indicador_3").alias("max_indicador_3"),
        spark_min("indicador_4").alias("min_indicador_4"),
        spark_max("indicador_4").alias("max_indicador_4"),
        spark_min("indicador_5").alias("min_indicador_5"),
        spark_max("indicador_5").alias("max_indicador_5"),
        spark_min("indicador_6").alias("min_indicador_6"),
        spark_max("indicador_6").alias("max_indicador_6")
    )
)

print("unificado", indicadores_unificado.count())
indicadores_unificado.show(5, truncate=False)
ref_minmax.show(truncate=False)

> 5 dataframes: ind_bairro, ind_favela, ind_municipio, ind_setor, indicadores_unifica...
unificado: 5863
+-----+-----+-----+-----+-----+-----+
|nível|id_recorte|mn_recorte|indicador_1|indicador_2|indicador_3|indicador_4|indicador_5|indicador_6|
+-----+-----+-----+-----+-----+-----+
|setor|310620005620016|310620005620016|100.0|99.24812030075188|100.0|1.1289158326797386|0.0|9.618296771864686|
|setor|310620005620017|310620005620017|95.4337899543379|100.0|99.54337899543379|1.2128689655172414|0.0|15.227286958683994|
|setor|310620005620041|310620005620041|100.0|100.0|99.1266375458514|1.1183932366723844|0.0|9.215841157436856|
|setor|310620005620052|310620005620052|100.0|98.4375|100.0|1.2916666666666667|0.0|8.065338638356823|
|setor|310620005620093|310620005620093|100.0|100.0|0.8656239623762376|0.0|6.165511611729379|
only showing top 5 rows
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|nível|min_indicador_1|min_indicador_2|min_indicador_3|max_indicador_2|min_indicador_3|max_indicador_4|min_indicador_5|max_indicador_6|min_indicador_5|max_indicador_6|max_indicador_6|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|setor|0.0|100.0|0.0|100.0|0.0|100.0|0.87377049188327869|1.0914893617821276|0.0|47.5|0.0|140.68896245874588|
|bairro|40.15151515151515|100.0|0.0|100.0|37.86487766998291|100.0|0.3095238952389523|1.5490654205687477|0.0|17.97752888988764|0.0|29.347823488735593|
|favela|15.08232526117647|100.0|0.0|100.0|15.432989765432989|100.0|0.308183279843120723|0.5722543354081156|0.0|12.97752888988764|0.0|47.08415110286383|
|municipio|99.33337380385186|99.33337380385186|97.12822380653677|97.12822380653677|99.36448058726858|99.36448058726858|0.62489213174313976|0.62489213174313976|1.9825673785155164|1.9825673785155164|3.863285188875807|3.863285188875807|

```

Figura 45 - Parte 2 do Código de construção da tabela de valores máximos e mínimos para a normalização.

Os indicadores 1, 2, 3, 4 e 6 foram normalizados conforme a fórmula abaixo, utilizando-se dos valores máximos e mínimos encontrados anteriormente.

$$VN_{ij} = 1 - \frac{MV_i - V_{ij}}{MV_i - mV_i}$$

VN_{ij} = valor normalizado na escala de 0 a 1 do indicador i no lugar j ;
 MV_i = maior valor obtido pelo indicador i entre todos os recortes geográficos pesquisados;

mV_i = menor valor obtido pelo indicador i entre todos os recortes geográficos pesquisados;

V_{ij} = valor obtido pelo indicador i no lugar j .

Contudo, para o cálculo do indicador 5, a fórmula da normalização será invertida, conforme destacado na introdução desta seção:

$$VN_{ij} = 1 - \frac{mV_i - V_{ij}}{mV_i - MV_i}$$

Importa frisar uma característica do cálculo de normalização do IDS: ele se utiliza do maior valor máximo e mínimo dentro todos os recortes. Por isso, nesse cálculo, é comum que um bairro seja normalizado pelo valor máximo ou mínimo que eventualmente tenha sido encontrado em um setor censitário.

Abaixo, a aplicação dessas fórmulas de normalização em código.

```
from pyspark.sql.functions import col, lit, min as spark_min, max as spark_max

# min e max de cada indicador em 10000 os recortes juntos
# setor, bairro, favela e município entram na mesma conta
limites = indicadores_unificado.agg(
    spark_min("indicador_1").alias("min_1"),
    spark_max("indicador_1").alias("max_1"),
    spark_min("indicador_2").alias("min_2"),
    spark_max("indicador_2").alias("max_2"),
    spark_min("indicador_3").alias("min_3"),
    spark_max("indicador_3").alias("max_3"),
    spark_min("indicador_4").alias("min_4"),
    spark_max("indicador_4").alias("max_4"),
    spark_min("indicador_5").alias("min_5"),
    spark_max("indicador_5").alias("max_5"),
    spark_min("indicador_6").alias("min_6"),
    spark_max("indicador_6").alias("max_6")
).collect()[0]

min_1, max_1 = limites["min_1"], limites["max_1"]
min_2, max_2 = limites["min_2"], limites["max_2"]
min_3, max_3 = limites["min_3"], limites["max_3"]
min_4, max_4 = limites["min_4"], limites["max_4"]
min_5, max_5 = limites["min_5"], limites["max_5"]
min_6, max_6 = limites["min_6"], limites["max_6"]

print("limites globais")
print("1:", min_1, max_1)
print("2:", min_2, max_2)
print("3:", min_3, max_3)
print("4:", min_4, max_4)
print("5:", min_5, max_5)
print("6:", min_6, max_6)

# normalizacao
# 1,2,3,4,6: VM = 1 - (max - V) / (max - min)
# 5: VM = 1 - (min - V) / (min - max)
# 5: alfabetismo alto vira nota baixa

def normaliza_ids(df):
    df = df.withColumn(
        "indicador_1_norm",
        lit(1) - (lit(max_1) - col("indicador_1")) / lit(max_1 - min_1)
    )
    df = df.withColumn(
        "indicador_2_norm",
        lit(1) - (lit(max_2) - col("indicador_2")) / lit(max_2 - min_2)
    )
    df = df.withColumn(
        "indicador_3_norm",
        lit(1) - (lit(max_3) - col("indicador_3")) / lit(max_3 - min_3)
    )
    df = df.withColumn(
        "indicador_4_norm",
        lit(1) - (lit(max_4) - col("indicador_4")) / lit(max_4 - min_4)
    )
```

Figura 46 – Parte 1 do código de normalização e verificação de valores globais de máximos e mínimos

```
    "indicador_4_norm",
    lit(1) - (lit(max_4) - col("indicador_4")) / lit(max_4 - min_4)
  )
}
df = df.withColumn(
  "indicador_5_norm",
  lit(1) - (lit(max_5) - col("indicador_5")) / lit(max_5 - min_5)
)

# indicador 5 invertido
df = df.withColumn(
  "indicador_5_norm",
  lit(1) - (lit(min_5) - col("indicador_5")) / lit(min_5 - max_5)
)

# IDS: média aritmética dos 6 normalizados
df = df.withColumn(
  "ids",
  (
    col("indicador_1_norm") +
    col("indicador_2_norm") +
    col("indicador_3_norm") +
    col("indicador_4_norm") +
    col("indicador_5_norm") +
    col("indicador_6_norm")
  ) / lit(6.0)
)
return df

indicadores["setor"] = normaliza_e_ids(indicadores["setor"])
indicadores["bairro"] = normaliza_e_ids(indicadores["bairro"])
indicadores["favela"] = normaliza_e_ids(indicadores["favela"])
indicadores["municipio"] = normaliza_e_ids(indicadores["municipio"])

> See performance \(1\) ✓ Checked for improvements

limites globais
1: 0.0 100.0
2: 0.0 100.0
3: 0.0 100.0
4: 0.02377802388327869 1.6914893617021276
5: 0.0 47.5
6: 0.0 148.68896245874588
```

Figura 47 – Parte 2 do código de normalização e verificação de valores globais de máximos e mínimos

Feita a gravação dos dados no *schema* Gold, procedeu-se a visualização de uma pequena amostra das tabelas, a fim de verificar os resultados: o IDS geral do município de Belo Horizonte, como pode ser visto na última imagem em sequência, é de 0.714, um valor alto, demonstrando bons níveis de desenvolvimento social.

Em consonância aos artigos publicados pelo Databricks, que preconizam que os dados nessa camada sejam modelados de forma agregada e desnormalizada, as tabelas foram gravadas nesse mesmo formato³.

³ A camada Gold é destinada à geração de relatórios e utiliza modelos de dados mais desnormalizados ([ver O que é a Arquitetura Medalhão?](#)).

```

3 days ago (20s) 43

# grava na gold
# cestas = so as variaveis do Censo
# indicadores = recorte + indicadores + ids

cestas["setor"].write.format("delta").mode("overwrite").saveAsTable("IDS_MVP.gold.cesta_variaveis_setor")
cestas["bairro"].write.format("delta").mode("overwrite").saveAsTable("IDS_MVP.gold.cesta_variaveis_bairro")
cestas["favela"].write.format("delta").mode("overwrite").saveAsTable("IDS_MVP.gold.cesta_variaveis_favela")
cestas["municipio"].write.format("delta").mode("overwrite").saveAsTable("IDS_MVP.gold.cesta_variaveis_municipio")

indicadores["setor"].write.format("delta").mode("overwrite").saveAsTable("IDS_MVP.gold.indicadores_setor")
indicadores["bairro"].write.format("delta").mode("overwrite").saveAsTable("IDS_MVP.gold.indicadores_bairro")
indicadores["favela"].write.format("delta").mode("overwrite").saveAsTable("IDS_MVP.gold.indicadores_favela")
indicadores["municipio"].write.format("delta").mode("overwrite").saveAsTable("IDS_MVP.gold.indicadores_municipio")

> See performance (8) ✓ Checked for improvements
    
```

Figura 48 - Código de gravação das tabelas na camada Gold.

display(spark.table("IDS_MVP.gold.indicadores_setor").select("CD_SETOR", "NM_BAIRRO", "NM_FCU", "indicador_1_norm", "indicador_2_norm", "indicador_3_norm", "indicador_4_norm", "indicador_5_norm", "indicador_6_norm", "ids").limit(10))

Ver desempenho (1) ✓ Verificado para melhorias

	CD_SETOR	NM_BAIRRO	NM_FCU	1.2 indicador_1_norm	1.2 indicador_2_norm	1.2 indicador_3_norm	1.2 indicador_4_norm	1.2 indicador_5_norm	1.2 indicador_6_norm	1.2 ids
1	310620005620016	Funcionários			0.9924812030075167		0.6472969811759815		0.96934768284905246	0.78468764450537...
2	310620005620017	Funcionários		0.954317899543378		0.9954337899543378	0.70364418372961		0.10839527359005742	0.793818524469544
3	310620005620041	Carmo				0.9912663755458514	0.6457381206997621		0.96553665567470279	0.78375685865338...
4	310620005620052	Santa Etelvânia			0.984375		0.753847851085238		0.85734008081818399	0.79909715198407...
5	310620005620091	Boa Viagem					0.564902399036799		0.84384863807661286	0.76812517200223...
6	310620005620222	Sion					0.7306440300763806		0.80737148791079574	0.80330258633119...
7	310620005620229	Sion					0.6844600832779573	0.8677124718410814	0.802278783692381174	0.78907522302023...
8	310620005620262	Sion		0.9794530547945206	0.9931598648215267		0.7605596521489147		0.857963577277778907	0.78843749485878...
9	310620005620293	Vila Santa Lúcia	Vila Bantagem Santa Lu...		0.9058629059829061	0.9829059829059829	0.213915133938123	0.8579782790309106	0.980320922278585862	0.68146720735608...
10	310620005620308	Santo Antônio					0.5976488355207632		0.04870886334121716	0.77439294981033

10 linhas 6.591s de runtime Atualizada há 5 min.

Figura 49 - Indicadores normalizados e IDS por setores censitários

Pós-Graduação em Ciência de Dados e Analytics
Sprint de Engenharia de Dados
Aluno: Daniel de Luna Marques
Matrícula: 4052026000228



Há 4 minutos (2)

```
display(
  spark.table("IDS_MVP_gold.indicadores_bairro")
  .select(
    "NM_BAIRRO",
    "indicador_1_norm",
    "indicador_2_norm",
    "indicador_3_norm",
    "indicador_4_norm",
    "indicador_5_norm",
    "indicador_6_norm",
    "ids"
  )
  .limit(10)
)
```

Ver desempenho (1) ✓ Verificado para melhorias

	NM_BAIRRO	1.2 indicador_1_norm	1.2 indicador_2_norm	1.2 indicador_3_norm	1.2 indicador_4_norm	1.2 indicador_5_norm	1.2 indicador_6_norm	1.2 ids
1	São Bento	0.9990962807448152	1	0.9990962807448152	0.730428384471448	0.9970049338362972	0.00683436108485836	0.80217735890965...
2	Padre Eustáquio	0.9905642514070579	0.9971692772221173	0.9991507831606353	0.419080204831808	0.9931859188452532	0.03007707625407141	0.73820608628782...
3	João Pinheiro	0.9993552546744036	0.9835589941972921	0.9993552546744036	0.32239536032711014	0.9718017445733852	0.022154667162782582	0.71643687926822...
4	Glória	0.9989611468938293	0.9968834400814877	0.9987533782725951	0.2907433237330428	0.9718810336009065	0.017167762187580538	0.71239838057157...
5	Álvoro Camargos	1	1	1	0.4784505833924705	1	0.0288011397525076	0.75120861622795...
6	Nova Granada	0.9924242424242425	0.9978354678354979	0.9963824963924964	0.4648773300137502	0.9897696340817334	0.039610911199501106	0.74631835199120...
7	Petropolis	0.999001990007984	0.9920159680638723	0.998003992015968	0.24747376742848937	0.9560624474151631	0.018462098075571975	0.70183671150117...
8	Minicratia	0.997927154278946	0.997334912644359	0.9985193959135327	0.270630819772097	0.9611810531633453	0.014916197442654822	0.7067515888915...
9	Coqueal	0.9955849889624724	0.971302428250706	0.9977824944812362	0.21793761769003772	0.817988739745184	0.008830660211890717	0.66823949426270...
10	Mariândia	1	0.9979209979209981	0.9979209979209981	0.2713395891869751	0.948184042042316	0.014087121493778665	0.70490879152116...

10 linhas 1,78s de runtime Atualizada há 4 min.

Figura 50 - Indicadores normalizados e IDS por bairros

Há 2 minutos (2)

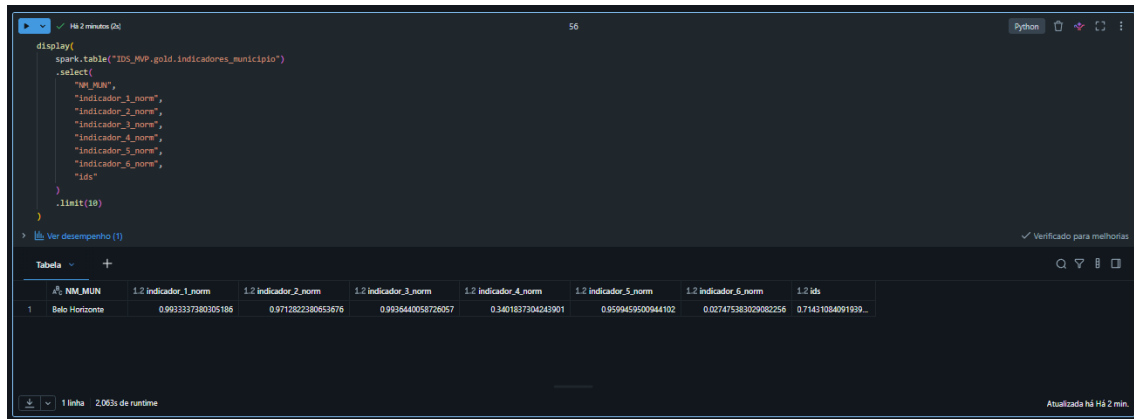
```
display(
  spark.table("IDS_MVP_gold.indicadores_favela")
  .select(
    "NM_FCU",
    "indicador_1_norm",
    "indicador_2_norm",
    "indicador_3_norm",
    "indicador_4_norm",
    "indicador_5_norm",
    "indicador_6_norm",
    "ids"
  )
  .limit(10)
)
```

Ver desempenho (1) ✓ Verificado para melhorias

	NM_FCU	1.2 indicador_1_norm	1.2 indicador_2_norm	1.2 indicador_3_norm	1.2 indicador_4_norm	1.2 indicador_5_norm	1.2 indicador_6_norm	1.2 ids
1	Paulo Freire	1	0.9648017699115044	0.9623008849537522	0.163488079583097338	0.86548079603486071	0.0070800219121248905	0.66719177003534...
2	Mariquinhas 2	1	0.9681016649152542	0.31355932203389825	0.1839889455136795	0.8165569218528797	0.0069585933422926486	0.64771750778214...
3	Vila Vera Cruz I	1	1	1	0.21225713053656139	0.856140350877193	0.0110302602951449	0.69665012308837...
4	Vila Independência II	0.9927797833935018	0.5234657039711191	1	0.18649076579972867	0.88992058806329549	0.008697420221007971	0.600220759008305...
5	Morro do Cabrito	0.9960442953020134	0.8389281744966443	0.983221470510067	0.21162162650986627	0.9238327360897707	0.008382921113463995	0.60043815510363...
6	Vila da Luz	0.9548351084812823	0.47731755424063116	0.7968441814595659	0.1949119770321599	0.8599237820578844	0.007563419600865375	0.54853267048306...
7	Vila Pietrangeli Vinda Nova	1	1	1	0.19932754033537559	0.8269646719538573	0.0097868692697672907	0.672880150831151
8	Pilar	1	0.9767441880405115	1	0.21082526325526405	0.9480181936322287	0.010602132833080817	0.69104162929451...
9	Antonio Ribeiro de Abreu	0.9318181818181818	0.5113636363636364	0.9886363636363636	0.2047900939405327	0.9059100205462305	0.009283112286091355	0.59196960241769...
10	Conjunto São Francisco de As...	1	1	1	0.2125388407401819	0.9546262365569481	0.009843263948051906	0.69615579005853...

10 linhas 1,807s de runtime Atualizada há 2 min.

Figura 51 - Indicadores normalizados e IDS por favelas ou comunidades urbanas



```
display(
  spark.table("IDS_MWP_gold.indicadores_municipio")
  .select(
    "NM_MUN",
    "indicador_1_norm",
    "indicador_2_norm",
    "indicador_3_norm",
    "indicador_4_norm",
    "indicador_5_norm",
    "indicador_6_norm",
    "ids"
  )
  .limit(10)
)
```

Ver desempenho [1] ✓ Verificado para melhorias

	NM_MUN	1.2 indicador_1_norm	1.2 indicador_2_norm	1.2 indicador_3_norm	1.2 indicador_4_norm	1.2 indicador_5_norm	1.2 indicador_6_norm	1.2 ids
1	Belo Horizonte	0.9933337380305186	0.9712822380653676	0.9936440058726057	0.3401837304243901	0.9599459500944102	0.027475383029082256	0.71431084091939...

1 linha 2.063s de runtime Atualizada há 2 min.

Figura 52 - Indicadores normalizados e IDS Geral para o município de Belo Horizonte

Com base nessa Modelagem, tornou-se exequível responder as perguntas de negócios enunciadas no título homônimo. Antes de realizar a etapa de Análise, no entanto, o Pipeline de Dados será explicado.

7. Pipeline de Dados

Todo o processo – desde a ingestão dos arquivos brutos do Censo Demográfico 2022 até a gravação das tabelas finais na camada Gold – foi implementado em um único Notebook dentro do ambiente Databricks, estruturado em blocos sequenciais que refletem a progressão da arquitetura medalhão: 1) ingestão e criação das tabelas Delta na camada Bronze, conforme apresentado no capítulo "Carga dos Dados"; 2) limpeza, padronização, tipagem e tratamento de valores nulos na camada Silver, conforme apresentado no capítulo "Qualidade de Dados"; e 3) consolidação das Cestas de Variáveis, cálculo dos indicadores e normalização na camada Gold, conforme apresentado no capítulo "Modelagem e Catálogo de Dados".

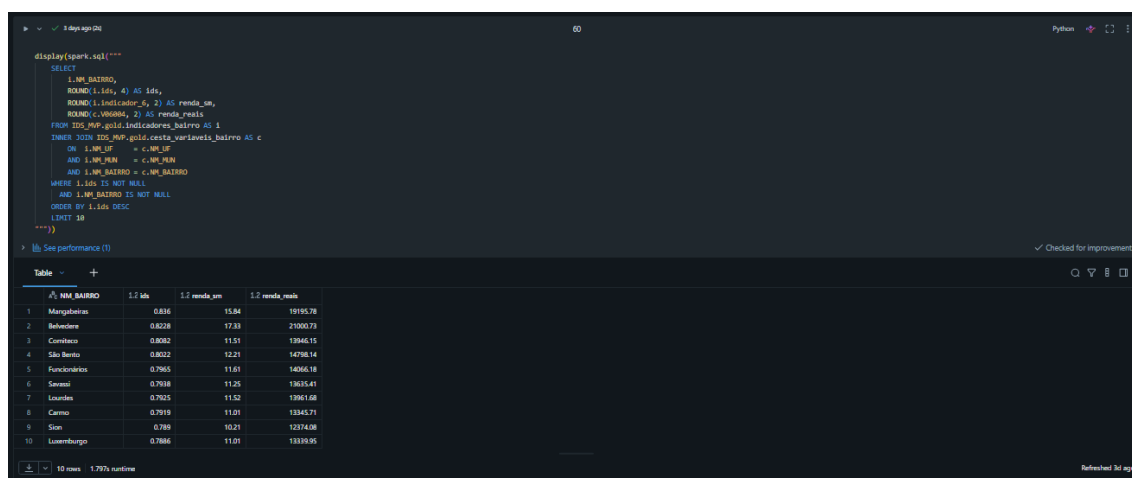
Optou-se por não repetir aqui os *screenshots* de código já exibidos nos capítulos anteriores, com o fim de evitar redundâncias que não agreguem valor a este trabalho. A evidência de que as tabelas de cada camada foram, de fato, gravadas (persistidas) na plataforma de nuvem já consta nas Figuras 27, 28 e 34, que apresentam o Overview dos *schemas* Bronze, Silver e Gold, respectivamente, com as tabelas Delta criadas em cada um deles.

Todo o código-fonte utilizado neste trabalho está disponibilizado publicamente no seguinte repositório do GitHub: https://github.com/danieldelunamarques-hub/puc_rio_mvp_engenharia_de_dados.

8. Análise de Dados

Passando por todas as etapas do processo ETL no escopo da arquitetura medalhão, resta analisar os dados gravados no *schema* Gold para responder as perguntas de negócios levantadas anteriormente e que os dados armazenados nessa camada possuem total condição de responder.

1. Quais são os 10 bairros com os maiores Índices de Desenvolvimento Social (IDS) em Belo Horizonte e a respectiva renda deles de acordo com o indicador 6 não-normalizado? Com base nessa tabela, analise a homogeneidade ou heterogeneidade desses bairros de acordo com a renda média dos bairros.



```
display(spark.sql("""
SELECT
    i_NM_BAIRRO,
    ROUND(i_ids, 4) AS ids,
    ROUND(i_indicador_6, 2) AS renda_sm,
    ROUND(c_valor6, 2) AS renda_real
FROM IDS_MRP_gold.indicadores_bairro AS i
INNER JOIN IDS_MRP_gold.cesta_variaveis_bairro AS c
ON i_NM_IDP = c_NM_IDP
AND i_NM_PEN = c_NM_PEN
AND i_NM_BAIRRO = c_NM_BAIRRO
WHERE i_ids IS NOT NULL
AND i_NM_BAIRRO IS NOT NULL
ORDER BY i_ids DESC
LIMIT 10
"""))
```

	i_NM_BAIRRO	i_ids	i_renda_sm	i_renda_real
1	Mangabeiras	0.836	15.34	19195.79
2	Bela Vista	0.828	17.33	21030.77
3	Corinthians	0.8043	11.51	13945.15
4	São Bento	0.8022	12.21	14798.14
5	Fundadores	0.7965	11.61	14506.18
6	Savassi	0.7938	11.25	13635.41
7	Lourdes	0.7925	11.52	13961.68
8	Carma	0.7919	11.01	13345.71
9	Sion	0.789	10.21	12374.08
10	Luxemburgo	0.7886	11.01	13339.95

Figura 53 - Código de consulta em SQL para responder a primeira pergunta.

Com base na tabela acima, é possível identificar que os bairros com os 10 maiores valores de IDS possuem, pelo menos, 10 salários mínimos (SM) em termos aproximados. Os valores máximos e mínimos correspondem, respectivamente, a 0.836 no bairro de Mangabeiras e aproximadamente 0.789 no bairro Luxemburgo.

Embora a tabela esteja ordenada pela variável IDS, do maior valor ao menor, é possível identificar que a variável renda não apresenta um comportamento de crescimento

constante em todos os casos. Por exemplo, embora o bairro Sion apresente o 9º valor de IDS, a renda média é menor ao nível imediatamente inferior do IDS de acordo com os bairros: Luxemburgo, que está em 10º lugar, com 11.01 SM, possui um rendimento médio superior a Sion, com 10.21 SM, embora esse último esteja em 9º lugar.

Isso ocorre, sobretudo, porque, sendo a média de 6 indicadores para cada um dos recortes ou subdivisões intramunicipais, o IDS torna-se um índice sensível a valores extremos, de maneira tal que a variável renda possa não ser a principal variável explicativa para todos os resultados do IDS.

Esse mesmo fenômeno ocorre ao longo da série analisada dos 10 bairros com maior IDS: entre Savassi (6º lugar) e Lourdes (7º lugar) esse fenômeno demonstra-se mais sutil, aquele bairro apresenta uma renda média expressa em SM (11.25) aproximadamente 6% menor que este último bairro (11.52 SM).

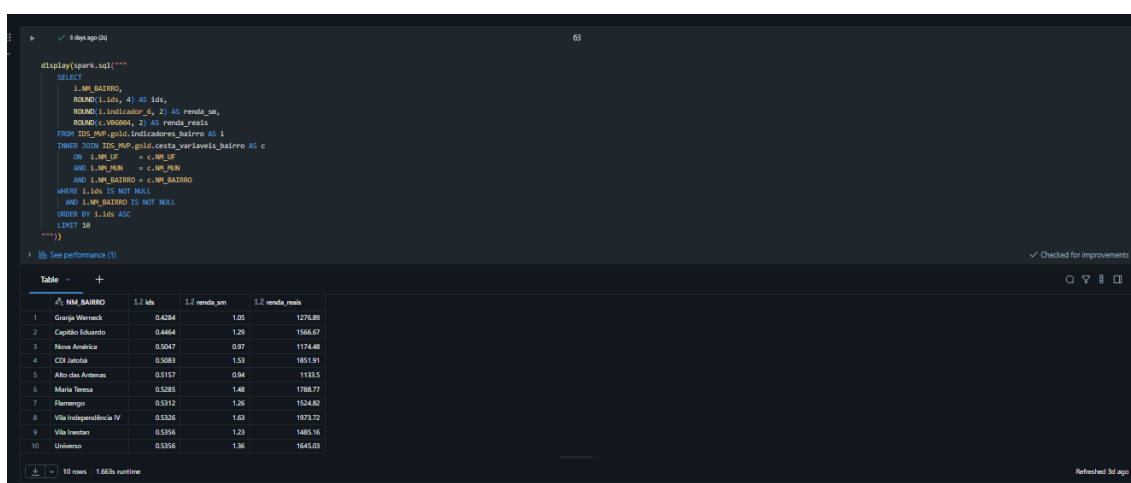
Outro exemplo nítido dessa desconexão entre renda pura e IDS geral ocorre no topo do ranking: o Belvedere apresenta o maior rendimento médio mensal de todo o grupo (17.33 SM, ou R\$ 21.000,73), superando o Mangabeiras (15.84 SM, ou R\$ 19.195,78). Ainda assim, o Mangabeiras ocupa o 1º lugar em função do desempenho relativo nos demais componentes.

Em relação à **homogeneidade vs. heterogeneidade** desse grupo:

- **Homogeneidade estrutural e de patamar social:** Sob a ótica do perfil macroeconômico, o grupo é **altamente homogêneo**. Todos os 10 bairros situam-se em uma faixa de renda consolidada de alta classe média e classe alta, com médias sempre superiores a 10 salários mínimos e IDS estritamente confinado entre 0.789 e 0.836 (uma amplitude de apenas ~0.047 pontos).
- **Heterogeneidade na distribuição da renda extrema:** Quando observada a distribuição interna de renda, nota-se uma **heterogeneidade significativa no estrato superior**: topo da tabela (Belvedere e Mangabeiras) descola dos demais, atingindo patamares entre 15 e 17 salários mínimos -- uma renda até 70% maior que a observada em bairros como Sion (10.21 SM) e Luxemburgo (11.01 SM).

Isso evidencia que, como o IDS captura a condição socioeconômica de uma região em perspectiva multidimensional, mesmo dentre os 10 bairros com maior rendimento, existe uma clara diferença social, percebida, nesta análise, na renda dos responsáveis.

- Quais são os 10 bairros com os menores Índices de Desenvolvimento Social (IDS) em Belo Horizonte e a respectiva renda deles de acordo com o indicador 6 não-normalizado?



```
display(spark.sql("""
SELECT
    i_nm_bairro,
    ROUND(i_idm, 4) AS ids,
    ROUND(i_indicador_6, 2) AS renda_nm,
    ROUND(c_valor_m, 2) AS renda_real
FROM tbc_mpf_gold.indicadores_bairro AS i
INNER JOIN tbc_mpf_gold.costa_variaveis_bairro AS c
ON i_nm_bairro = c_nm_bairro
AND i_nm_bairro = c_nm_mpf
AND i_nm_bairro = c_nm_bairro
WHERE i_idm IS NOT NULL
AND i_nm_bairro IS NOT NULL
ORDER BY i_idm ASC
LIMIT 10
"""))
```

	i_nm_bairro	i_idm	i_indicador_6	c_valor_m
1	Granja Werneck	0,4284	1,05	12712,03
2	Capitão Eduardo	0,4464	1,29	15046,07
3	Nova América	0,5047	0,87	11744,48
4	CDJ Jardim	0,5083	1,53	18211,91
5	Alto das Antenas	0,5157	0,94	11935,5
6	Maria Teresa	0,5285	1,48	17888,77
7	Flamengo	0,5312	1,26	15248,02
8	Vila Independência IV	0,5326	1,63	19773,72
9	Vila Inestan	0,5356	1,23	14851,16
10	Universo	0,5356	1,36	16451,03

Figura 54 - Código de consulta em SQL para responder a segunda pergunta.

O resultado revela um grupo com pontuações situadas entre 0,4284 (Granja Werneck) e 0,5356 (Vila Inestan e Universo), configurando uma amplitude de 0,1072 ponto. Todas as localidades desse grupo enquadram-se na faixa categórica média de desenvolvimento social (0,40 a 0,60), com destaque para Granja Werneck (0,4284) e Capitão Eduardo (0,4464), que se distanciam dos demais e formam o estrato mais crítico da escala de bairros da capital. Do terceiro colocado em diante, os índices convergem gradativamente para patamares superiores a 0,50, estendendo-se de Nova América (0,5047) até Universo (0,5356).

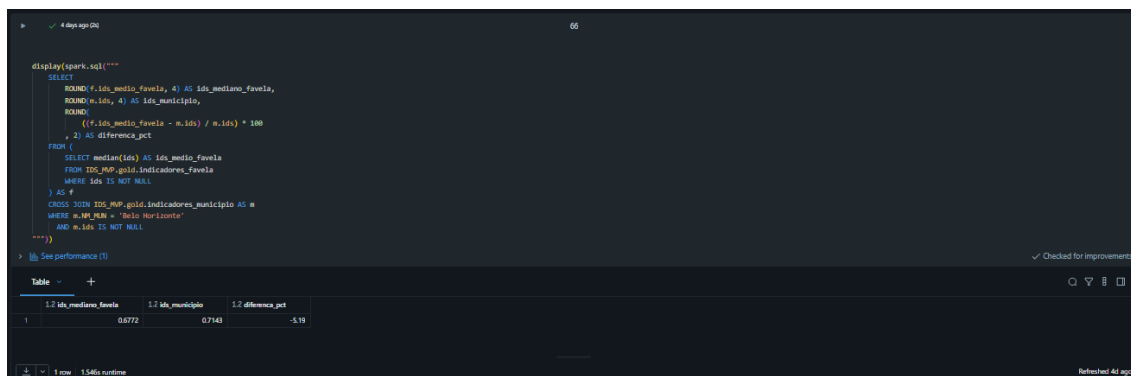
No que diz respeito ao indicador de rendimento médio nominal mensal dos responsáveis (indicador 6 não-normalizado), observa-se um padrão severamente

comprimido, oscilando entre 0,94 e 1,63 salário mínimo (R\$ 1.133,50 a R\$ 1.973,72), com média global de 1,27 salário mínimo (R\$ 1.542,10). Dois territórios apresentam médias inferiores a um salário mínimo nacional vigente em 2022 (R\$ 1.212,00): Alto das Antenas, com 0,94 SM (R\$ 1.133,50), e Nova América, com 0,97 SM (R\$ 1.174,48). No extremo oposto desse conjunto, Vila Independência IV e CDI Jatobá registram os maiores rendimentos médios, atingindo 1,63 SM (R\$ 1.973,72) e 1,53 SM (R\$ 1.851,91), respectivamente.

A ordenação pelo IDS evidencia que a variável de renda não dita isoladamente a classificação de vulnerabilidade, demonstrando na prática o caráter multidimensional do índice. O bairro Granja Werneck, detentor do menor IDS geral (0,4284), apresenta renda média de 1,05 SM (R\$ 1.276,89), montante superior ao observado em Alto das Antenas (0,94 SM) e Nova América (0,97 SM), situados em posições superiores no ranking (0,5157 e 0,5047).

Sob a perspectiva da homogeneidade versus heterogeneidade, constata-se uma forte homogeneidade socioeconômica de base: todos os bairros estão confinados no estrato de baixa renda e subsistência, auferindo rendimentos que representam uma fração mínima daqueles verificados nos bairros de maior IDS da capital (onde as médias variam entre 10 e 17 salários mínimos).

3. Quão percentualmente maior ou menor é o IDS mediano das favelas ou comunidades urbanas em relação ao IDS municipal?



```
display(spark.sql("""
SELECT
  ROUND(f.id_medio_favela, 4) AS id_mediano_favela,
  ROUND(m.id, 4) AS id_municipio,
  ROUND(
    ((f.id_medio_favela - m.id) / m.id) * 100
  ) AS diferenca_pct
FROM (
  SELECT median(ids) AS id_medio_favela
  FROM IDS_PMP_gold.indicadores_favela
  WHERE ids IS NOT NULL
) AS f
CROSS JOIN IDS_PMP_gold.indicadores_municipio AS m
WHERE m.muni_plm = "Belo Horizonte"
AND m.id IS NOT NULL
"""))
```

	1.2 id_mediano_favela	1.2 id_municipio	1.2 diferenca_pct
1	0.6772	0.7143	-5.19

Figura 55- Código de consulta em SQL para responder a terceira pergunta.

A consulta revela que o IDS mediano das favelas e comunidades urbanas de Belo Horizonte é 0.6772, enquanto o IDS do município é 0.7143, resultando em uma defasagem de -5.19%.

Dessa comparação, destacam-se os seguintes pontos:

- Distância relativa contida: a diferença de pouco mais de 5% em relação ao índice municipal indica que a infraestrutura básica urbana (cobertura de abastecimento de água, energia e coleta de resíduos) possui ampla capilaridade na capital, atenuando a queda nos componentes de saneamento do IDS.
- Uso da mediana vs. média: A escolha pela mediana do conjunto das favelas neutraliza eventuais distorções provocadas por comunidades isoladas de IDS extremamente atípico (seja muito baixo ou muito alto), refletindo com precisão o patamar central vivido pela maior parte das comunidades urbanas.

Se, no entanto, dentro dos 10 bairros com maior IDS existem diferenças de natureza socioeconômica, essas seriam mais acentuadas nas favelas, considerando que esse é um tipo de território marcado pelas diferenças sociais. A próxima pergunta poderá responder a isso em mais detalhes.

4. Alocando o IDS das favelas ou comunidades urbanas em faixas categóricas, analise a concentração percentual dos registros nessas faixas.

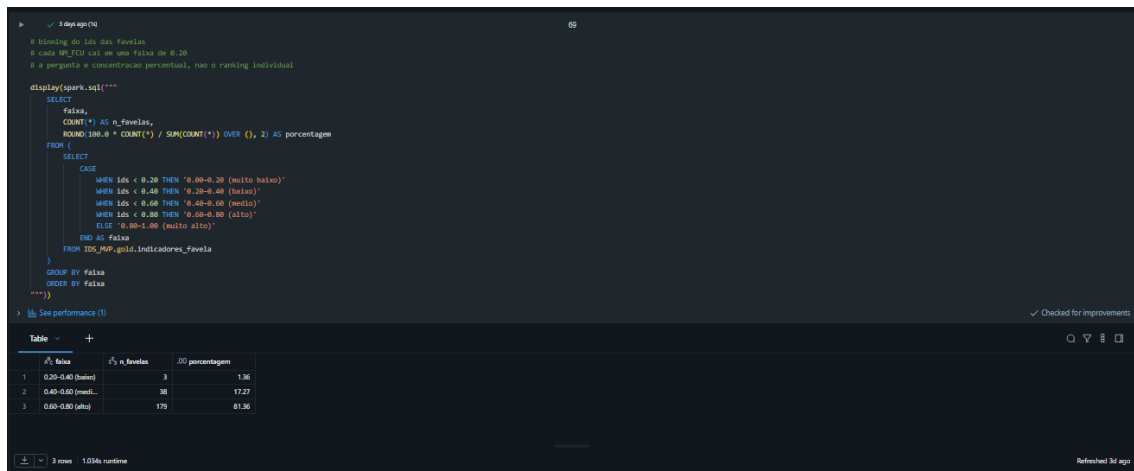


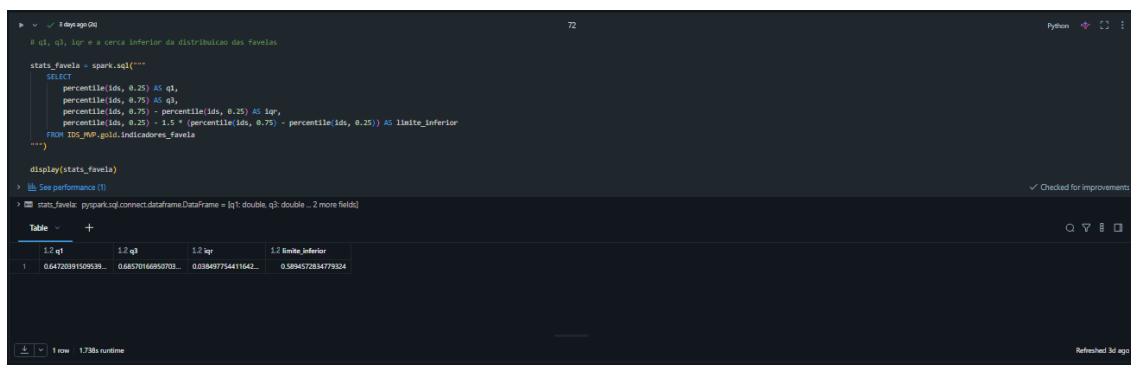
Figura 56- Código de consulta em SQL para responder a quarta pergunta.

A distribuição das 220 favelas e comunidades urbanas de Belo Horizonte demonstra uma acentuada concentração na faixa de IDS considerado alto (0.60 a 0.80), abrangendo 179 comunidades, o que representa expressivos 81.36% do total de registros analisados. Na sequência, a faixa de nível médio (0.40 a 0.60) reúne 38 territórios, correspondendo a 17.27% da amostra, enquanto a faixa de nível baixo (0.20 a 0.40) compreende apenas 3 localidades, perfazendo residuais 1.36%. Nenhuma comunidade foi classificada nas faixas extremas de muito baixo (inferior a 0.20) ou muito alto (superior a 0.80).

Esse comportamento revela que, longe de apresentarem uma dispersão caótica ao longo de toda a régua de desenvolvimento social, as favelas da capital exibem uma notável convergência em torno do patamar mediano de 0.6 observado anteriormente. A quase total ausência de registros na faixa inferior a 0.40 e a inexistência de registros abaixo de 0.20 confirmam que a infraestrutura urbana básica e os serviços essenciais alcançam a esmagadora maioria desses assentamentos, estabelecendo um piso mínimo de qualidade de vida que evita situações generalizadas de indigência estrutural.

É importante destacar que não existem favelas cujo IDS esteja situado na faixa "muito alto". Contudo, esse cenário pode se mostrar diferente quando os setores censitários forem analisados.

5. Dentre a distribuição do IDS das favelas ou comunidades urbanas, quais estão abaixo do limite inferior?



```

# q1, q3, iqr e a cerca inferior da distribuição das favelas

stats_favela = spark.sql("""
SELECT
    percentile(ids, 0.25) AS q1,
    percentile(ids, 0.75) AS q3,
    percentile(ids, 0.75) - percentile(ids, 0.25) AS iqr,
    percentile(ids, 0.25) - 1.5 * (percentile(ids, 0.75) - percentile(ids, 0.25)) AS limite_inferior
FROM IDS_MVP_gold.indicadores_favela
""")

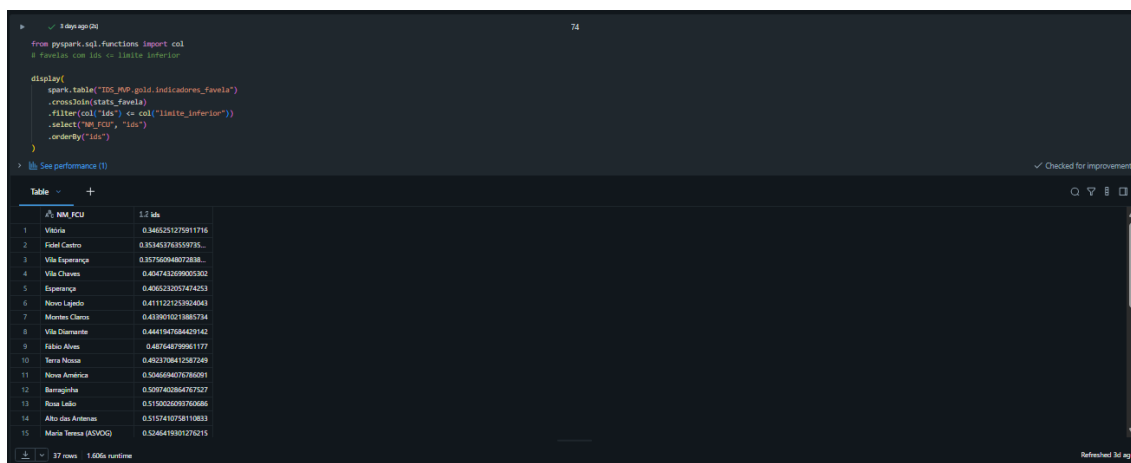
display(stats_favela)

```

	1.2 q1	1.2 q3	1.2 iqr	1.2 limite_inferior
1	0.647203915209539...	0.685701666950703...	0.038497754411642...	0.5894572834779334

Figura 57 – Parte 1 do código de consulta em SQL para responder a quinta pergunta.

Com o resultado acima, obtive-se o resultado necessário para identificar quais favelas possuem o IDS igual ou menor ao limite inferior encontrado:



```

from pyspark.sql.functions import col
# favelas com ids <= limite inferior

display(
    spark.table("IDS_MVP_gold.indicadores_favela")
    .crossJoin(stats_favela)
    .filter(col("ids") <= col("limite_inferior"))
    .select("NM_FCU", "ids")
    .orderBy("ids")
)

```

	1.2 NM_FCU	1.2 ids
1	Vitória	0.3465251272611716
2	Fidel Castro	0.353453783559978...
3	Vila Esperança	0.357525948072838...
4	Vila Chaves	0.4047433299003302
5	Esperança	0.4065232057474253
6	Novo Lajedo	0.411221253924043
7	Montes Claros	0.4339010213885734
8	Vila Diamante	0.444194768429142
9	Fábio Alves	0.4871646799961177
10	Terra Nova	0.4921708412587249
11	Novo América	0.5046034276786091
12	Barragem	0.509740284797977
13	Rua Lado	0.5150203261765666
14	Alto das Antenas	0.5157410758110833
15	Maria Teresa (ASVOG)	0.5246419391276215

Figura 58 - Parte 2 do código de consulta em SQL para responder a quinta pergunta.

São 37 favelas e comunidades urbanas de Belo Horizonte apresentam Índice de Desenvolvimento Social (IDS) igual ou inferior ao limite inferior estatístico calculado

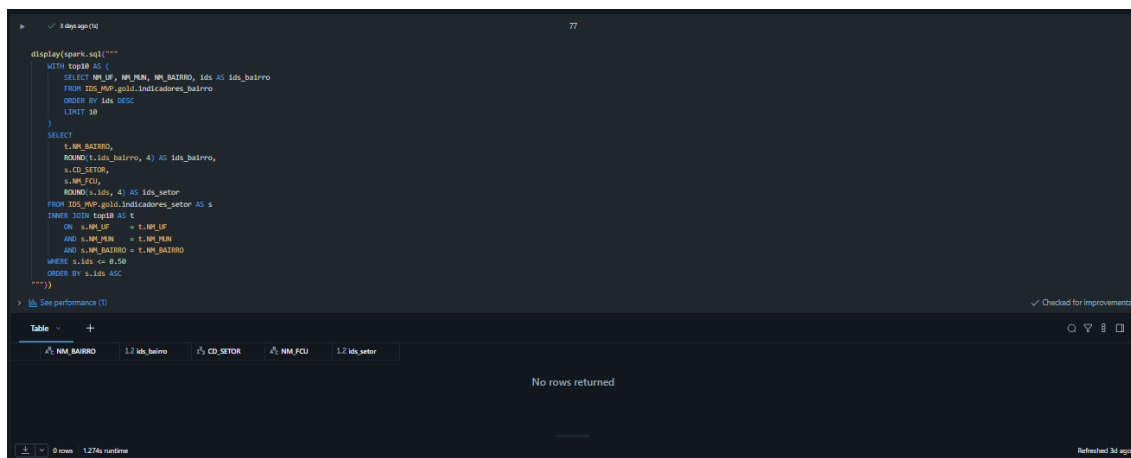
(0,5895), correspondendo a 16,82% do total das 220 comunidades mapeadas no município.

Dessa forma, essas 37 localidades configuram discrepantes estatísticos à esquerda (outliers negativos), afastando-se expressivamente da faixa central consolidada na qual se posiciona a maioria das comunidades da capital (como visto anteriormente, de 80% das favelas estão situadas na faixa que compreende um IDS de 0.6 a 0.8).

A leitura integral desse contingente revela uma clara gradação interna de vulnerabilidade. No ponto mais crítico da distribuição situam-se as três únicas comunidades classificadas no estrato de nível baixo (IDS inferior a 0,40): Vitória (0,3465), Fidel Castro (0,3535) e Vila Esperança (0,3576). Em seguida, desponta um segmento intermediário de severidade composto por sete localidades com índices situados entre 0,40 e 0,50, abrangendo Vila Chaves (0,4047), Esperança (0,4065), Novo Lajedo (0,4111), Montes Claros (0,4339) etc.

O contraste entre a menor pontuação observada (Vitória, com 0,3465) e a localidade de maior valor entre os discrepantes (Vila Canto do Sabiá, com 0,5807) resulta em uma amplitude interna de 0,2341 pontos, evidenciando que as privações estruturais não se distribuem de modo homogêneo nem mesmo entre as periferias mais fragilizadas. Enquanto 81,36% das comunidades de Belo Horizonte convergem para a faixa de IDS alto (0,60 a 0,80), este grupo de 37 territórios representa bolsões onde as carências socioeconômicas e urbanas persistem de forma aguda, elegendo-os como foco prioritário para eventuais intervenções por meio de políticas públicas.

6. Existem setores censitários em situação de vulnerabilidade crítica ($IDS < 0,50$) situados dentro dos 10 bairros de maior IDS?



```
display(spark.sql("""
WITH top10 AS (
    SELECT NM_UF, NM_MUN, NM_BAIRRO, IDS AS IDS_bairro
    FROM IDS_MUP_gold.indicadores_bairro
    ORDER BY IDS DESC
    LIMIT 10
)
SELECT
    t.NM_BAIRRO,
    ROUND(t.IDS_bairro, 4) AS IDS_bairro,
    s.CD_SETOR,
    s.NM_SETOR,
    ROUND(s.IDS, 4) AS IDS_setor
FROM IDS_MUP_gold.indicadores_setor AS s
INNER JOIN top10 AS t
ON s.NM_UF = t.NM_UF
AND s.NM_MUN = t.NM_MUN
AND s.NM_BAIRRO = t.NM_BAIRRO
WHERE s.IDS < 0.5
ORDER BY s.IDS ASC
"""))
```

NM_BAIRRO	IDS_BAIRRO	CD_SETOR	NM_SETOR	IDS_SETOR
-----------	------------	----------	----------	-----------

No rows returned

0 rows 1.276s runtime

Figura 59 - Código de consulta em SQL para responder a sexta pergunta.

Dentre os bairros com maiores IDS, não existem setores censitários com IDS menor que 0.5, o que indica uma maior homogeneidade quando esses bairros são analisados pela ótica dos setores censitários.

7. Considerando as mesmas faixas categóricas de IDS, qual é a distribuição percentual dos setores censitários situados na malha urbana regular em comparação com os de favelas e comunidades urbanas?

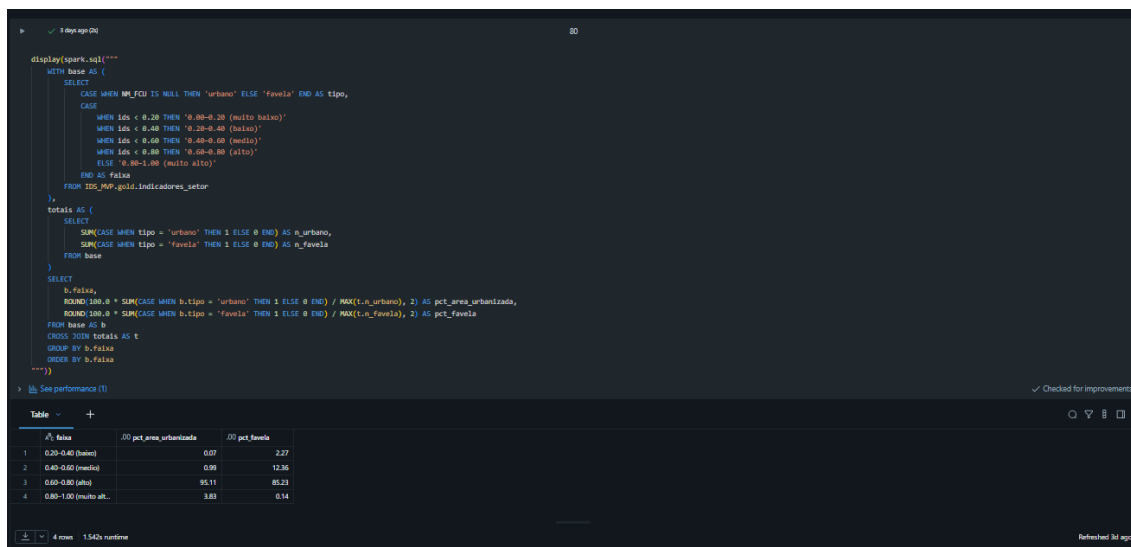


Figura 60 - Código de consulta em SQL para responder a sétima pergunta.

A distribuição dos setores censitários revela que a expressiva maioria do território de Belo Horizonte converge para a faixa de IDS alto (0,60 a 0,80), patamar que abriga 95,11% dos setores da malha urbana regular e 85,23% dos setores situados em favelas e comunidades urbanas. Notavelmente, a faixa de IDS muito baixo (0,00 a 0,20) não registrou nenhuma ocorrência em qualquer uma das tipologias territoriais.

A clivagem socioespacial entre as duas realidades manifesta-se com vigor nas faixas de menor desenvolvimento social (abaixo de 0,60). Nas favelas, os estratos médios (12,36%) e baixo (2,27%) somam 14,63% dos setores, proporção quase 14 vezes superior à verificada na área urbanizada regular, que acumula apenas 1,06% nessa situação de vulnerabilidade (0,99% em nível médio e residuais 0,07% em nível baixo).

9. Autoavaliação

Pessoal e profissionalmente, o desenvolvimento deste MVP representou tanto um desafio quanto uma experiência enriquecedora: na avaliação do autor, o trabalho foi atingiu satisfatoriamente os objetivos traçados em elaborar, de ponta a ponta em ambiente de nuvem, o pipeline para o cálculo do Índice de Desenvolvimento Social (IDS) de Belo Horizonte com base nos dados do Censo Demográfico de 2022.

O aprendizado exigiu bastante dedicação, sendo o principal obstáculo o uso do PySpark: enfrentar cada erro de execução ao longo do código foi o que possibilitou alcançar a segurança técnica até a entrega deste trabalho.

Finalmente, encerro esse projeto satisfatoriamente, crendo que o trabalho foi feito no limite das minhas capacidades atuais de entregar a melhor, mais completa e funcional solução a uma problemática que, inclusive, faz parte das minhas atribuições profissionais e de pesquisa.

Em termos de evolução técnica na Engenharia de Dados, uma continuidade natural desse MVP seria a automação do fluxo do cálculo do IDS e tratamento de variáveis do Censo Demográfico de 2022. Pessoalmente, conheço esses dados por trabalhar com eles rotineiramente, de maneira tal que, os problemas encontrados na etapa de Qualidade, por exemplo, não são os únicos: a título de exemplo, além dos valores X, os dados de renda do município do Rio de Janeiro apresentam registros com “.” em seus valores. Tal característica não foi levada em conta no escopo deste projeto por dois motivos: 1) os dados concernentes a Belo Horizonte apresentaram apenas os valores anonimizados “X”; 2) não acho que seria apropriado, ao menos para os meus interesses particulares para este projeto, deixar que um conhecimento prévio sobre a natureza dos dados antecipasse um problema que não havia sido encontrado nos dados efetivamente tratados e analisados.

Ademais, esse projeto poderia ser enriquecido através da adição de dados geoespaciais – em algumas linguagens eles são denominados como variáveis geométricas – a fim de que o desenvolvimento social possa ser analisado visualmente em seu comportamento espacializado.

Grato pela boa vontade dos professores e, sobretudo, pelo tempo investido, que espero poder abrilhantar a caminhada acadêmica e científica de muitos alunos, bem como certamente abrilhantar a minha.

Daniel de Luna.